

Software Requirements Specification

for

Exam Software Application

Version 1.0

Prepared by

Group Name: *Dystopians*

Şevval Naz Özkaraca
Ahmet Deniz Sezgin
Barış Bağbekleyen
İrem Ayrancınar
Engin Erkurt
Rana Neslihan Kurt
Mert Özek

2300008192
2300005633
2300005352
2300005623
2300006437
2300006496
2300007579

2300008192@stu.iku.edu.tr
2300005633@stu.iku.edu.tr
2300005352@stu.iku.edu.tr
2300005623@stu.iku.edu.tr
2300006437@stu.iku.edu.tr
2300006496@stu.iku.edu.tr
2300007579@stu.iku.edu.tr

Instructor: Akhan Akbulut

Course: Software Engineering

Lab Section: *Tuesday, 14.00*

Teaching Assistant: *Büşra Kocaçınar Öz*

Date: 19.05.2026

Contents

CONTENTS	II
REVISIONS	III
1 INTRODUCTION	1
1.1 PROJECT PURPOSE AND SCOPE, AND OBJECTIVES	1
1.2 ROLES AND RESPONSIBILITIES	1
1.3 TECHNICAL ASSUMPTIONS AND CONSTRAINTS	1
1.4 NAMING CONVENTIONS	1
2 REQUIREMENTS	2
2.1 FUNCTIONAL REQUIREMENTS	2
2.2 NON FUNCTIONAL REQUIREMENTS	2
3 OTHER REQUIREMENTS	3
4 SYSTEM ARCHITECTURE AND ARCHITECTURAL DESIGN	4
4.1 LOGICAL VIEW	4
4.2 DEPLOYMENT VIEW	4
4.3 USE CASE VIEW	4
5 DESIGN AND IMPLEMENTATION	5
6 OTHER SUPPORTING INFORMATION	6
7 DEPLOYMENT AND OPERATION GUIDE	7
8 DESIGN DECISIONS – BEFORE AND AFTER	8
9 REFERENCES	9

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Şevval Naz Özkaraca, Barış Bağbekleyen, Ahmet Deniz Sezgin, Engin Erkurt, Rana Neslihan Kurt, İrem Ayrıncı, Mert Özek	Final integration release of the Software Requirements Specification. Functional and non-functional requirements consolidated; design decisions captured under a dedicated Before / After section with screenshots from the integrated build; deployment guide added for Windows 10/11 classroom hosts; references list extended with additional academic-monitoring and supervision tools.	19.05.2026

1 Introduction

The Network-Based Laboratory Exam System (referred to throughout this document as the System or NBLES) is a self-contained client-server platform designed to supervise paper-replacement examinations in a shared computer laboratory. Unlike cloud-hosted proctoring services, the System runs entirely on the classroom local area network: a single server process coordinates the exam, while one client process per student handles authentication, local monitoring, incident reporting and the final upload. No student data leaves the institutional network. The remainder of this document follows the IEEE 830 outline. Section 1 establishes the project purpose, the team responsibilities, the assumptions under which the System operates and the naming conventions used throughout. Section 2 enumerates the functional and non-functional requirements organised by area. Section 3 covers additional requirements such as persistence files, internationalisation, privacy and reuse objectives. Section 4 presents the system architecture and three architectural views. Section 5 documents design and implementation notes. Section 6 contains supporting information including sample payloads and packaging notes. Section 7 records the principal design decisions as a side-by-side Before / After narrative accompanied by visual evidence from the integrated build. Section 8 concludes with the deployment and operation guide, and Section 9 lists the academic and technical references consulted during the project.

1.1 Project Purpose and Scope, and Objectives

The exam monitoring application is specifically designed to prevent, detect, and report instances of cheating or external assistance, particularly during laboratory sessions. The system monitors the user's computer activities, browser tabs, and applications, recording them in real time. This information is displayed on the proctor's screen in a list customized for each student. The proctor can identify prohibited applications and websites; if a student attempts to access these restricted sites or applications, the proctor can observe the student, determine if a violation has occurred, and terminate the exam. This system can also be automated to prevent selected applications and

windows from opening, or to automatically close them if they do open. The proctor can execute commands to intervene in the student's exam. The proctor can create new rules at any time.

1.2 Roles and responsibilities

The project is developed by seven students working on distinct but coordinated areas. The team's contact details are listed below, followed by the architectural responsibilities of each member. Each member is the primary author and integration point for the modules listed below.

Member	Primary Responsibility	Owned Modules and Components
Ahmet Deniz Sezgin	Client runtime, monitoring pipeline, and submission flow	client/client/ws_client.py, incidents.py, transfers.py, submission.py, exam.py, gui.py, incident_buffer.py, outbound_buffer.py, and the client-side custommodules
Engin Erkurt	Service discovery, port management, and reconnect-safe buffering	common/discovery.py, common/discovery_v2.py, common/server_ports.py, and client/client/outbound_buffer.py
Barış Bağbekleyen	Server runtime, exam lifecycle, and admin command handling	server/server/app.py, main.py, handlers.py, tasks.py, state.py, session_state.py, shutdown.py, submissions.py, ip_guard.py, settings_service.py, auth_validation.py; server-side regression test suite
Şevval Naz Özkaraca	Secure transmission, authentication preflight, and CATS helper	common/security.py, common/protocol.py, common/text_safety.py, common/ipc_ws.py, client/school_service.py, client/auth_util/check_user.cpp, auth_util/ad_auth.py
İrem Ayranpınar	Main dashboard user interface	server/server/ui/dashboard_tk.py, dashboard_dialogs_tk.py, dashboard_table_helpers.py, and the dashboard popups module
Rana Neslihan Kurt	Reusable Qt UI components and visual styling	server/server/ui/dashboard_qt.py, policy_settings_qt.py, row_refresh.py, common_ui/styles.py, common_ui/widgets.py, common_ui/theme.py, common_ui/tk_theme.py

Mert Özek	Persistence layer and audit trail across the system	Eight-state session model (server/server/session_state.py), data/server/server_users.json + incidents.jsonl persistence, reconstruct-timeline utilities, and event-protocol coherence between client and server through common/events.py
-----------	---	--

1.3 Technical Assumptions and Constraints

1.3.1 Operating System and Platform

The system targets Windows 10 and Windows 11 (x86_64) on both the server and client machines. The screen-replay subsystem depends on Windows-only desktop capture APIs (gdigrab), and the focused-window incident detector relies on Win32 titlebar APIs; no Linux or macOS support is in scope for Version 1.0.

1.3.2 Programming Languages

Python 3.13 is the primary language for the server, client, shared protocol and test suite. The May_12 deployment split organises the code into two independent bundles; each bundle ships its own, its own requirements.txt and its own offline-packages folder. The script enforces the TARGET_PYTHON = (3, 13) check at install time and refuses to proceed on any other interpreter version. A small amount of native code supports the optional Active Directory fallback path on the client; the main authentication route uses pure Python through client/school_service.py.

1.3.3 External Tools and Runtime Dependencies

FFmpeg is invoked by the client's replay_recorder to produce a rolling 60-second screen replay using the gdigrab capture device, and to stitch cached segments into a single MP4 (with MPEG-TS fallback) at submission time. The runtime looks up FFmpeg in three ordered locations: the EXAM_FFMPEG_PATH environment variable first, then , then the system PATH. The client Python dependencies are aiohttp, psutil, cryptography, requests, beautifulsoup4 and PySide6; the server Python dependencies are aiohttp, psutil, cryptography and PySide6. The Tk back-end uses the standard-library tkinter module and therefore does not require an extra package. Both bundles can be installed entirely offline from a pre-built wheelhouse under offline-packages/wheelhouse, verified through a manifest.sha256 integrity file.

1.3.4 Network and Deployment

The system is intended for a single-LAN classroom deployment in which the operator's server machine and all student client machines reside on the same Layer-2 broadcast domain. Service discovery is implemented over UDP broadcast on port 5354 using the magic identifier 6064-SERVER and is augmented with subnet-directed broadcast, an optional multicast group, and VPN-aware interface enumeration so that the beacons still reach the clients when an overlay adapter such as ZeroTier, Tailscale, or WireGuard is present. The data channel is WebSocket over TCP on the configured server port with application-layer HMAC signing and Fernet encryption. The server binds to a non-privileged TCP port and is protected by an IP allow-list; a duplicate-server

probe and a background guard prevent two instances from binding to the same server identifier on the same LAN. The May_12 client and server bundles are physically separated: the client folder contains no server package and the server folder contains no client package, and the May_12 VALIDATION.md verifies this boundary.

1.3.5 Authentication

User authentication is delegated to the institution's CATS portal: client/school_service.py logs in against <https://cats.iku.edu.tr/portal/relogin> using requests and BeautifulSoup, then compares the scraped student identifier with the one entered locally to prevent impersonation. The server side additionally restricts admission to identifiers listed in server/allowed_users.json; the operator edits this file before an exam. An Active Directory path is available as a fallback through client/auth_util/ad_auth.py and auth_util/check_user.cpp, controlled by ad_domain in server/auth_config.json. The system maintains no local password database. A runtime-only temporary-disable workflow (/disablecatsauth, /approveauth) lets the operator manually approve logins when the CATS service is unreachable.

1.3.6 Storage

Persistent state is stored as a small set of JSON and JSONL files under each role's data/ tree; the system does not require a separate database server. The server-side files are: data/server/server_users.json (the registered-user table, including session state, time spent, ban flag, kick count, last action, and submission metadata); data/server/incidents.jsonl (append-only incident history); data/server/exam_policy.json (normalised exam policy bundle); data/server/process_blacklist.txt (line-oriented blacklist); data/server/process_definitions.json (process database definitions used for family-of-executables matching); and data/server/incident_rules.json (normalised incident-rule database with the statuses unknown, whitelist, warning, blacklist). The client-side state is organised per session UUID under data/client/{uuid}/, including buffer/{connect_timestamp}/packets.jsonl for the durable outbound queue, recordings/cache/ for the rolling MPEG-TS replay segments, recordings/replays/ for the stitched on-demand replays, and exam_files/ for the downloaded exam materials. Editable persistence files are written through an atomic temp-file-then-rename pattern so that a process crash cannot leave a half-written file; append-only layers tolerate a single truncated last line.

1.3.7 Standards Followed

The SRS is written following IEEE 830 and ISO/IEC/IEEE 29148 for software requirements specification. The WebSocket transport conforms to RFC 6455; the auxiliary HTTP endpoints conform to RFC 7230; cryptographic primitives follow RFC 6234 (SHA-2).

1.3.8 Assumptions

(1) FFmpeg is reachable through EXAM_FFMPEG_PATH, through , or through the system PATH. (2) All machines have synchronised system clocks within approximately ± 2 seconds for HMAC replay-window validation. (3) The classroom network does not employ client-isolation features that would block intra-LAN traffic between the student and proctor machines. (4) Students do not have administrator privileges on their client machines, which is required for the policy engine to terminate prohibited processes. (5) Python 3.13 is installed manually on each machine before runs.

1.4 Naming Conventions

1.4.1 Document Formatting Conventions

Italic angle-bracketed text (<like this>) marks template guidance and placeholders that must be replaced before final submission. Bold marks keywords introduced for the first time. Monospace marks source-code identifiers, file paths, command names, configuration keys, and protocol field names. Acronyms are expanded on first use (e.g., Software Requirements Specification (SRS)) and then abbreviated. Section numbering uses hierarchical Arabic numerals to a maximum depth of three levels.

1.4.2 Requirement Identifiers

Functional and non-functional requirements use the prefix scheme <TYPE>-<AREA>-<NN>, where TYPE is FR (functional) or NFR (non-functional), AREA is a two- to four-letter code for the functional area, and NN is a two-digit zero-padded sequence within that area.

The functional-area codes are summarised below.

Area code	Functional area
FR-SRV	Server lifecycle, route registration, admin commands, and command dispatch.
FR-CLI	Client lifecycle, discovery, login, exam preparation, and timer UI.
FR-MON	Activity monitoring (process, focused window, hardware, idle, replay).
FR-INC	Incident classification, persistence, and decision flow.
FR-POL	Policy database, normalisation, broadcast, and editing.
FR-REC	Reconnect and durable buffering of incidents and evidence.
FR-SUB	Submission packaging, multipart upload, and artifact transfer.
FR-AUTH	Authentication preflight, temporary disable, and admin validation.
FR-IPC	Loopback inter-process communication (manager — CLI — GUI).
FR-UI	Operator dashboard and student timer user interfaces.
FR-PROJ	Read-only projector page and Server-Sent Events feed.
FR-INS	Offline Windows installer and bundled wheelhouse.
FR-DB	Persistence files and on-disk schema.
FR-AUDIT	Audit trail and timeline reconstruction.

1.4.3 Source-Code Conventions

The codebase follows PEP 8 for Python and standard Qt-style conventions for the C++ helper. Modules and files use; packages use a single lowercase word; classes use PascalCase; functions, methods, and variables use snake_case; module-level constants use UPPER_SNAKE_CASE; private (module-internal) names take a leading underscore. Test files are named test_<unit_under_test>.py. WebSocket and IPC message types use dotted lowercase namespaces (e.g., timer.command, incident.report). Log tags in console output use bracketed uppercase prefixes such as [INCIDENT], [RECORDER], [EXAM], and [ERROR].

The non-functional-area codes are summarised below.

Area code	Non-functional area
NFR-PERF	Performance and latency budgets.
NFR-SEC	Security controls (transport, authentication, input).
NFR-PRIV	Privacy controls (data minimisation, projector exposure).
NFR-REL	Reliability and crash-safety.
NFR-UX	Usability of operator and student interfaces.
NFR-OBS	Observability — logs, audit traces, error reporting.
NFR-OPS	Operations, deployment, and offline rebuildability.
NFR-PORT	Portability across machines and Python versions.
NFR-MAINT	Maintainability of source organisation and tests.

1.4.4 User-Interface Conventions

Window titles use Title Case (e.g., Exam Timer, Finish Exam, Operator Dashboard). Action buttons use imperative Title Case (e.g., Choose File, Upload And Finish, Request Start). Status messages addressed to the student are written in second person, sentence case, and end with a period. Operator-side commands begin with a forward slash and a lowercase verb (e.g., /startexam, /finishexam, /ban, /kick, /disablecatsauth).

2 Requirements

2.1 Functional Requirements

2.1.1 Server Runtime (FR-SRV-*)

FR-SRV-01 The server shall validate CLI arguments at startup (host, port, interval, exam duration, upload limits) and shall perform a duplicate-server probe over the discovery channel, refusing to boot on invalid arguments or if another server with the same id is already reachable on the LAN.

FR-SRV-02 The server shall expose the HTTP routes /health, /auth/status, /login, /exam/config, /exam/files, /exam/submission, /client/artifact, /projector, /projector/events, and the WebSocket route /ws.

FR-SRV-03 The server shall load persistent user state from disk on startup, normalise legacy fields, create any missing default files so a fresh checkout boots cleanly, and re-save the cleaned-up state.

FR-SRV-04 The server shall maintain an eight-state session state machine (WAITING, RUNNING, ADMIN_PAUSED, DISCONNECTED_PAUSED, VIOLATION_PAUSED, AWAITING_SUBMISSION, SUBMITTED, BANNED) with a single mutation entry point.

FR-SRV-05 The server shall broadcast a time tick every broadcast_interval seconds (default one second) and shall advance per-student time-spent counters atomically.

FR-SRV-06 The server shall expose a centralised admin command dispatcher shared by the CLI stdin and the dashboard IPC, covering exam control (/startexam, /finishexam, /pauseexam, /resumeexam, /addtime, /forgiveviolation), user moderation (/kick, /ban, /unban, /killpid), policy management (/editblacklist, /applyblacklist, /editpolicy, /applypolicy, /editdefinitions, /applydefinitions, /exportsettings, /importsettings, /remembersettings), and monitoring / status requests (/savescreen, /clients, /exam, /help).

FR-SRV-07 The server shall remain protocol-compliant: WebSocket close messages shall respect the 125-byte control-frame limit without splitting multi-byte UTF-8 sequences, and the banned flag on every banned user shall be preserved across a global finish-exam operation.

2.1.2 Client Runtime (FR-CLI-*)

FR-CLI-01 The client shall locate the server through automatic UDP discovery or through an explicit host / port command-line argument.

FR-CLI-02 The client shall authenticate by POST /login using the operator-configured login id and credentials, and shall present any HMAC token configured by the operator.

FR-CLI-03 The client shall fetch the exam configuration and exam files only after a successful login and shall extract the exam ZIP into a manifest-tracked managed folder, rejecting absolute paths, parent-directory traversal and symbolic links during extraction.

FR-CLI-04 The client shall display the remaining exam time to the student in the Tk or Qt timer window and shall synchronise with the server tick.

FR-CLI-05 The client shall keep all local monitors running across transient WebSocket disconnects: only an explicit shutdown shall stop monitoring, and the server shall treat reconnect as a network condition while preserving the user's session state.

2.1.3 Authentication (FR-AUTH-*)

FR-AUTH-01 The server shall validate user credentials through an external identity preflight: client/school_service.py shall scrape the CATS portal at the configured URL and shall verify that the scraped student identifier matches the one supplied locally; the Active Directory path through auth_util/check_user.cpp shall be used as a fallback when ad_domain is configured in server/auth_config.json. Server admission shall additionally be restricted to the identifiers listed in server/allowed_users.json.

FR-AUTH-02 The server shall support a runtime-only temporary-disable workflow for CATS or AD with a default 60-second TTL and a server-clamped maximum, never persisted to policy files.

FR-AUTH-03 The server shall maintain an admin-managed credential-validation queue (/authrequests, /approveauth, /denyauth) that records the login identifier, IP and disabled-auth modes but never the submitted password.

FR-AUTH-04 The server shall reject login attempts from users in BANNED, KICKED, SUBMITTED or FINISHED state with a clear error.

2.1.4 Monitoring (FR-MON-*)

FR-MON-01 The client shall sample the running-process list at a configurable interval and emit a structured snapshot to the server.

FR-MON-02 The client shall detect focused-window changes with Unicode-normalised window titles and emit a focused-window event.

FR-MON-03 The client shall detect idle behaviour against a configurable threshold and emit an idle-warning event when crossed.

FR-MON-04 The client shall capture a hardware delta (USB devices, removable media) whenever the OS reports a change.

FR-MON-05 The client shall record a rolling 60-second screen replay using FFmpeg and shall save the replay on demand from the server.

2.1.5 Incident Rules and Policy (FR-INC-*, FR-POL-*)

FR-INC-01 Incidents shall be classified by the normalised incident-rule engine as unknown, whitelist, warning or blacklist; a whitelist match shall suppress the incident before any warning or blacklist evaluation.

FR-INC-02 Each incident shall be persisted with a server-stamped `server_received_at`, the originating `login_id`, the originating `client_id`, the matched rule and the configured-actions list.

FR-POL-01 The operator shall edit and apply the process blacklist, the exam policy and the process definitions through the dashboard or through dedicated slash-commands; every successful apply shall broadcast the new policy version to all connected clients.

FR-POL-02 When a violation-severity incident opens and the rule's `auto_violation_pause` flag is true, the server shall transition the affected student to `VIOLATION_PAUSED` with a pinned remaining-time snapshot.

2.1.6 Submission and Artifact Transfer (FR-SUB-*)

FR-SUB-01 The client shall build a submission bundle containing the student submission, a runtime process report, a hardware snapshot, a focused-window report and the latest screen replay.

FR-SUB-02 The bundle shall include a `manifest.json` with per-file SHA-256 checksums and an `archive_sha256` over the whole archive; the server endpoint shall accept multipart uploads in any field order and shall verify both the per-file and archive checksums before accepting the submission.

FR-SUB-03 The server shall accept submission uploads after a global finish-exam operation even if the user state still resolves to `WAITING`.

2.1.7 Reconnect and Buffering (FR-REC-*)

FR-REC-01 Unsent incidents shall be persisted to a durable on-disk buffer with monotonically increasing sequence numbers.

FR-REC-02 On reconnect, the buffered-incident restore path shall deduplicate by sequence identity so that the server never sees the same incident twice.

2.1.8 Local IPC and UI (FR-IPC-*, FR-UI-*)

FR-IPC-01 The local IPC channel shall bind only to 127.0.0.1, shall reject any connection that does not present the per-process random token passed through environment variables, and shall fall back to stdin for client and server CLI processes when no IPC peer is connected.

FR-UI-01 The instructor dashboard shall offer feature parity between the Tk and Qt backends for operator-critical surfaces (student list, alerts, exam control, policy management), built from a reusable UI library that supplies alert cards with severity variants, status badges for connection / security / violation states, timeline rows and summary table components shared between the live dashboard and post-exam review screens.

2.1.9 Projector View (FR-PROJ-*)

FR-PROJ-01 The /projector HTTP route shall serve a read-only aggregate notification surface for in-room display, streaming updates through /projector/events using Server-Sent Events so an instructor can mirror the projector view in any HTML5 browser without requiring a desktop UI on the projection machine.

Offline Installer (FR-INS-*)

FR-INS-01 The bundle shall provide a one-line setup entry point that installs the Python dependencies from the local wheelhouse under offline-packages/wheelhouse without any network access.

FR-INS-02 The setup script shall enforce TARGET_PYTHON = (3, 13) and shall refuse to proceed on any other interpreter version, printing the detected version and exiting with a non-zero status.

FR-INS-03 The offline bundle shall verify the integrity of bundled files against the included manifest.sha256 before installing.

FR-INS-04 The runtime shall locate FFmpeg in this order: the EXAM_FFMPEG_PATH environment variable, then the bundled offline-packages/ffmpeg/bin folder, then the system PATH.

FR-INS-05 The launcher batch scripts (run_*_qt.bat, run_*_tk.bat, run_*_cli.bat) shall prefer py -3.13 and shall fall back to python only when it resolves to Python 3.13.

2.1.10 Persistence Schema and API (FR-DB-*)

FR-DB-01 The server shall persist exam-session and per-student data into JSON and JSONL files under data/server/ (server_users.json for the user table, incidents.jsonl for the incident history, exam_policy.json for the active policy, incident_rules.json for the rule database, process_definitions.json for the process database and process_blacklist.txt for the legacy blacklist). The schema is created on first run and normalised on every subsequent start.

FR-DB-02 The persistence layer shall expose backward-compatible read and write functions for the server lifecycle (load_server_state on startup, save_server_state on every state change, and incident-append helpers) and shall guarantee that concurrent writers cannot corrupt a file: editable files are rewritten atomically through a temp file and a rename, and append-only files tolerate a single truncated last line on recovery.

2.1.11 Audit Trail (FR-AUDIT-*)

FR-AUDIT-01 The audit-trail store shall maintain five layers rooted at data/audit/: an immutable session header (sessions/<uuid>.json), an append-only raw-event stream (events/<uuid>.jsonl), an interpreted-violation layer with explicit links_raw_event_ids pointers (violations/<uuid>.jsonl), a per-day

instructor-action log (instructor_actions/<YYYY-MM-DD>.jsonl) and a replace-in-place review-decision layer with full revision history (reviews/<uuid>.json).

FR-AUDIT-02 The reconstruct_timeline(session_uuid) query shall merge all five layers into a chronologically ordered list and shall prefer the observation timestamp over the recording timestamp.

FR-AUDIT-03 The audit-trail HTTP surface shall expose GET /audit/timeline, POST /audit/review and GET /audit/sessions, each gated by the RBAC chokepoint with the actions INCIDENT_READ and INCIDENT_REVIEW_DECISION.

2.2 Non Functional Requirements

2.2.1 Performance Requirements

NFR-PERF-01 Message delivery latency on the lab LAN shall be bounded: the end-to-end latency between an instructor command issued in the dashboard and the corresponding state change observed by the target client shall not exceed two seconds; an incident emitted by the client shall reach the server and be persisted to incidents.jsonl within 500 milliseconds.

NFR-PERF-02 The server time-broadcast tick shall fire at one-second intervals with a drift of no more than 100 milliseconds across a 45-minute exam.

NFR-PERF-03 The system shall sustain at least 35 concurrent WebSocket sessions without dropping or reordering messages on a single server host with 8 GB RAM and a quad-core CPU.

NFR-PERF-04 A submission upload of up to 200 MB shall complete within five minutes on a 100 Mbit/s switched LAN, with progress feedback shown to the student.

NFR-PERF-05 The reconstruct_timeline(session_uuid) query shall complete within two seconds for a session with up to 10,000 raw-event lines.

2.2.2 Safety and Security Requirements

NFR-SEC-01 All sensitive LAN events (exam_policy, policy_update, session_state, incident_report, kill_process, pause_exam, resume_exam) shall be HMAC-signed and optionally Fernet-encrypted, and shall be subject to a server-enforced replay window.

NFR-SEC-02 The local IPC channel shall be loopback-only, token-protected and shall reject second connections to prevent cross-process command injection.

NFR-SEC-03 Untrusted input shall be handled safely: exam ZIP extraction shall reject absolute paths, parent-directory traversal and symbolic links and shall confine extraction to the manifest-tracked managed folder; submission and artifact uploads shall be size-limited (--max-submission-mb, --max-artifact-mb) and verified by SHA-256 against the manifest before being accepted.

2.2.3 Software Quality Attributes

2.2.3.1 - Reliability

NFR-REL-01 The system shall fail closed on connectivity loss: if the network connection to the server drops, the client shall continue local monitoring and buffering rather than relaxing restrictions.

NFR-REL-02 Editable persistence files (including session headers and review decisions in the audit-trail store) shall be written through an atomic temp-file-then-rename pattern so a process crash cannot leave a half-written file. Append-only layers shall tolerate a single truncated last line on a crash and skip it during load.

2.2.3.2 - Portability

NFR-POR-01 The runtime shall be deployable from a single requirements.txt install on Windows 10 / 11 without requiring an external database, message broker or container runtime.

2.2.3.3 - Maintainability

NFR-MNT-01 Source code shall be organised into four top-level Python packages (common/, server/, client/, client/custommodules/) with no circular imports.

NFR-MNT-02 Every new feature shall ship with at least one automated test covering the happy path and at least one covering an error path.

2.2.3.4 - Observability

NFR-OBS-01 Every monitor shall write its events to a per-monitor JSONL log and every CLI process shall capture stdout and stderr to a per-process file for post-mortem analysis.

2.2.3.5 - Usability

NFR-USA-01 The operator dashboard shall present every per-student action as a one-click or single-command operation; complex flows shall not exceed three clicks from the student row to completion.

3 Other Requirements

3.1 Persistence Files

The system uses JSON and JSONL files as its persistence layer; there is no external database server. The schema is initialised on first run by the relevant server modules and is normalised on every subsequent start so a fresh checkout boots cleanly. The server-side persistence files live under `data/server/` and the client-side persistence files live per session UUID under `data/client/{uuid}/` as described in Section 1.3.6. Append-only files (incidents.jsonl, the durable outbound buffer packets.jsonl, and the per-monitor JSONL logs) use one JSON object per line with a stable monotonically increasing sequence number where applicable. Editable files (server_users.json, exam_policy.json, incident_rules.json, process_definitions.json) are rewritten through an atomic temp-file-then-rename pattern, and missing default values are repaired on load so that legacy snapshots remain readable.

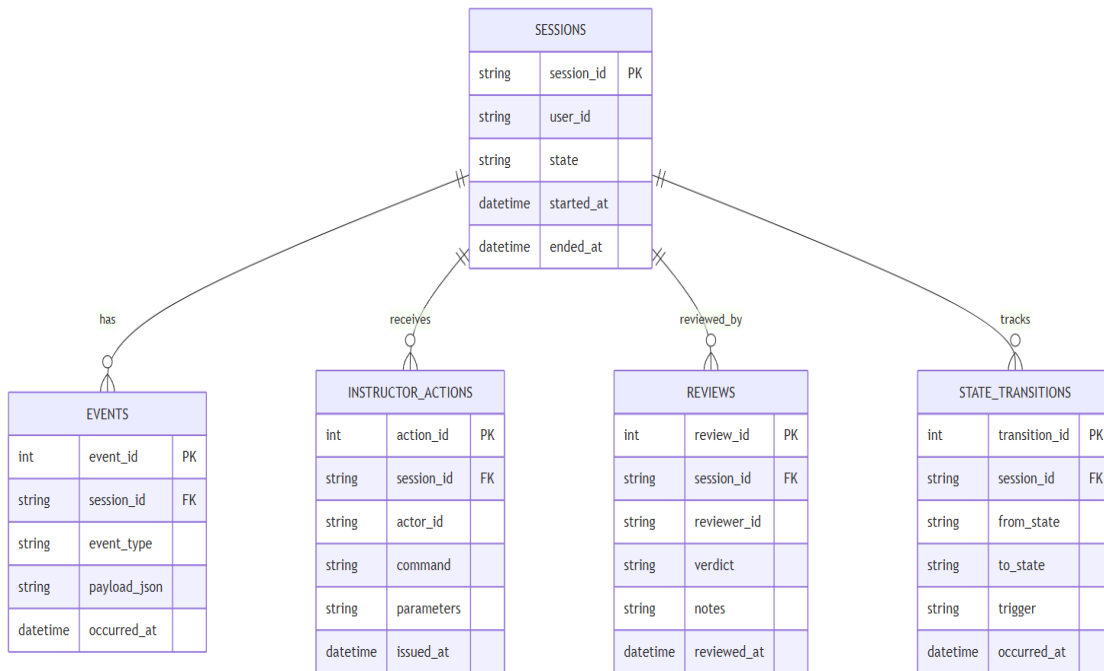


Figure 3-1 — Persistence file layout for the system (server JSON files under `data/server/` and per-session client JSONL tree under `data/client/{uuid}/`).

3.2 Internationalisation Requirements

All persistence layers write UTF-8 with `ensure_ascii=False` so Turkish characters (ş, ç, ğ, ı, ö, ü) in user names, session reasons, instructor commands and review notes are preserved verbatim. The dashboard UI strings are externalised so that a future translation file can replace the English defaults without code changes.

3.3 Privacy and Legal Requirements

All student-identifying data (login id, IP, hostname, screen replay, focused-window titles) is stored only on the institution-controlled server and never transmitted to a third party. The projector view exposes only public-safe aggregates. Submission archives and evidence files are retained according to the institution's exam-records retention policy and are deleted at the end of that period.

3.4 Reuse Objectives

The shared common/ package and the reusable UI component library are designed for reuse beyond the immediate exam scenario: the audit-trail layers and the session state machine could support a lab-attendance system, and the dashboard UI library could host an academic-integrity review console as a separate application.

4 System Architecture and Architectural Design

NBLES is structured as an N-tier architecture in which a stateless presentation layer (the dashboard, the student timer GUI and the projector page) communicates with a stateful application layer (the server's HTTP and WebSocket handlers, the admin command dispatcher and the time broadcaster) over a secure transport layer. The application layer delegates persistence to JSON and JSONL files under `data/server/` and `data/client/{uuid}/`, and delegates authentication to the institutional CATS portal and an optional Active Directory helper.

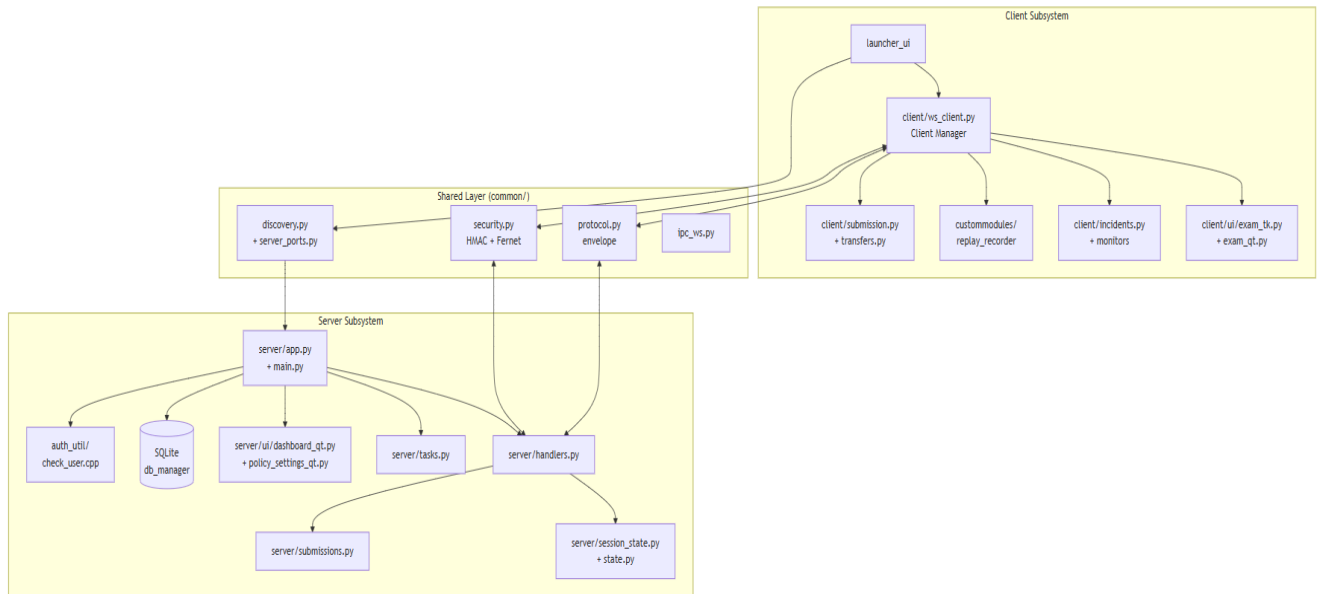


Figure 4-1 — Component diagram of the NBLES subsystems and their interconnections.

4.1 Logical View

The logical view of NBLES is presented through three complementary UML diagrams. The class diagram in Figure 4-2 enumerates the principal domain classes — the server, the WebSocket client, the incident monitor, the replay recorder, the submission bundle, the session-state machine, the security envelope, and the database manager — together with their public operations and associations. Figure 4-3 shows the eight-state session model used by the server to track each participant; transitions are triggered either by proctor commands or by automatic events (auto-pause on incident, timeout, upload success). Figure 4-4 expresses the activity flow of the client-side incident detection pipeline, from raw monitor samples through policy matching to evidence collection and server reporting.

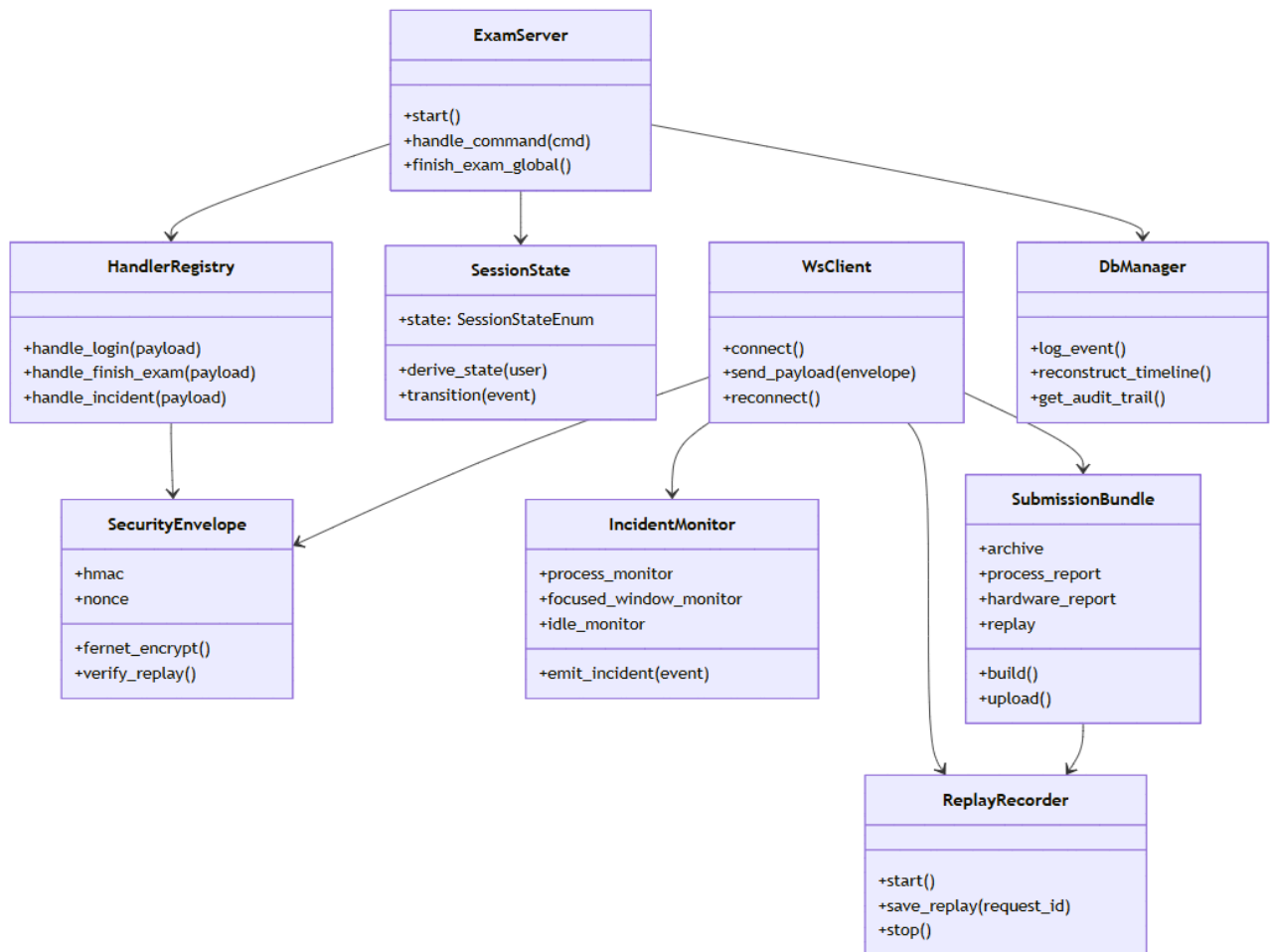


Figure 4-2 — Class diagram of the principal NBLES domain classes.

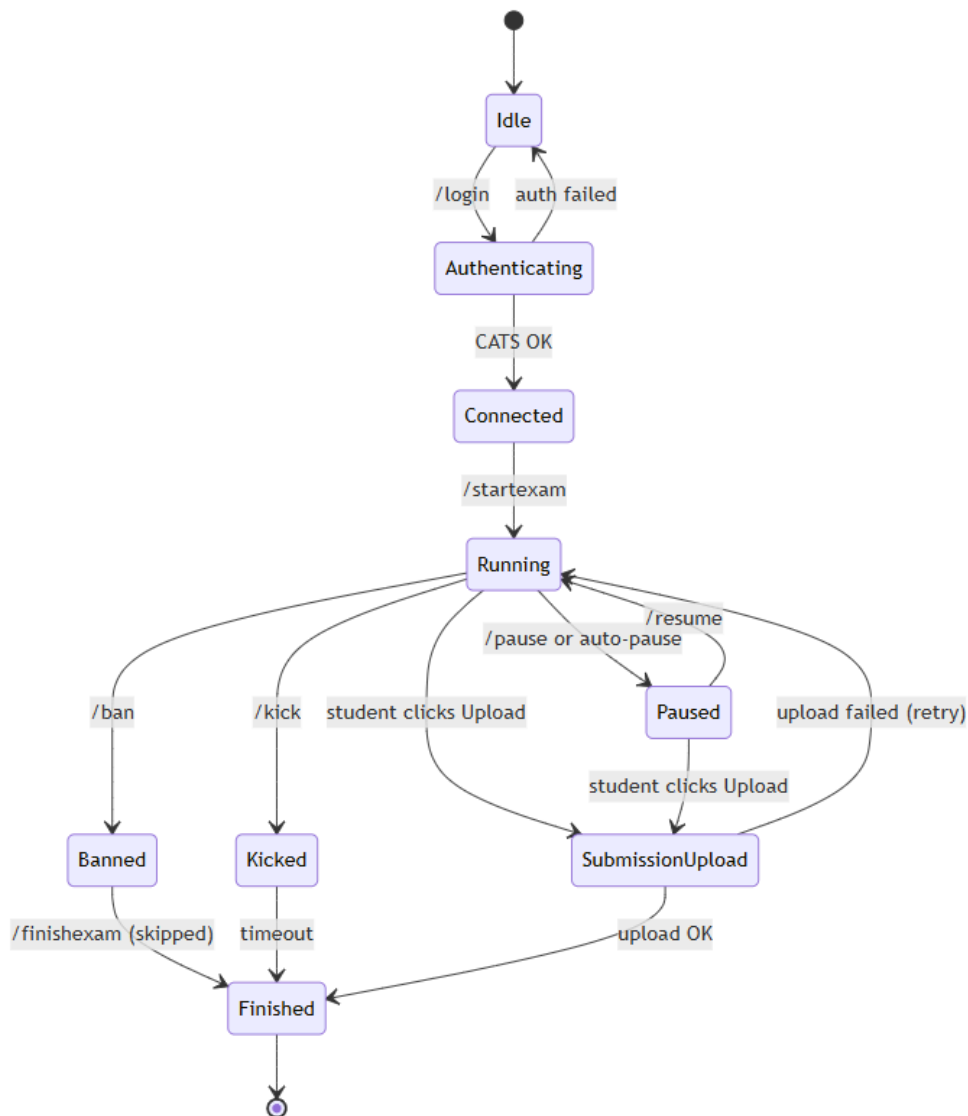


Figure 4-3 — State diagram of the eight-state session model.

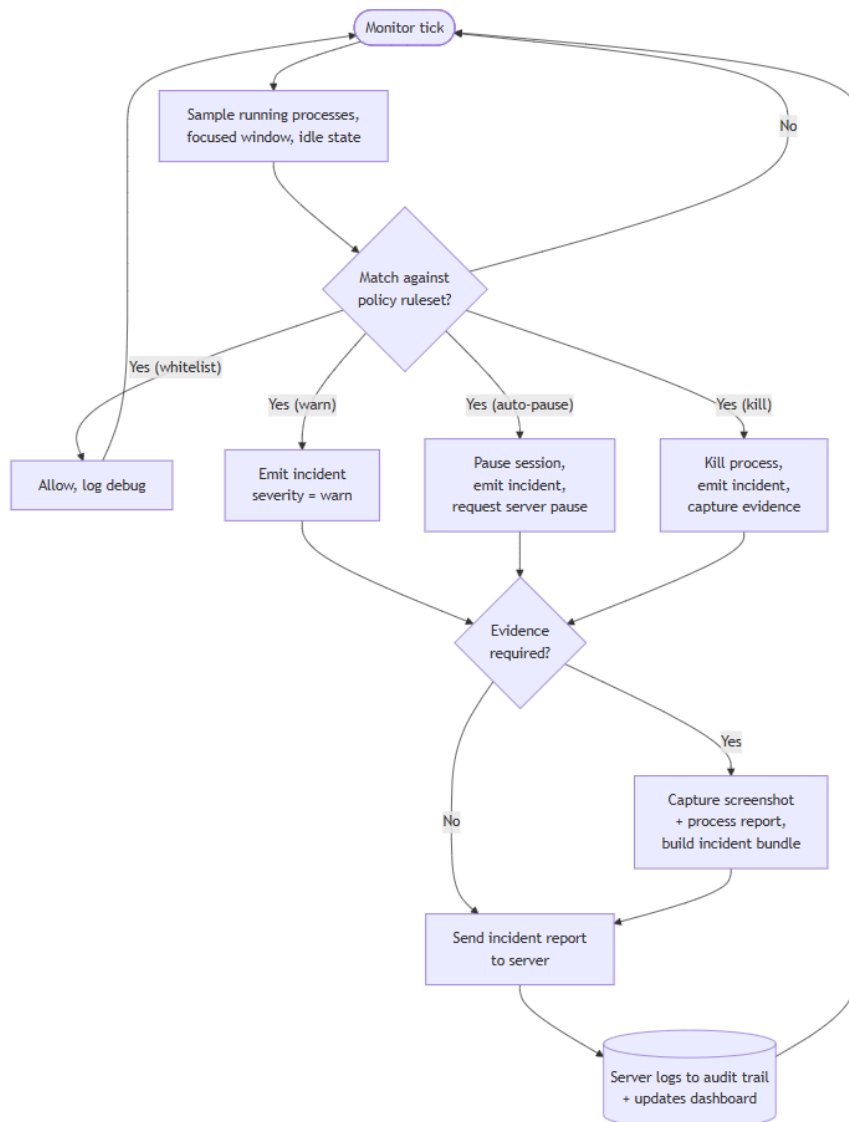


Figure 4-4 — Activity diagram of the client-side incident detection pipeline.

4.2 Deployment View

NBLES is deployed on a single classroom LAN that shares a Layer-2 broadcast domain. The deployment is split into two independent bundles. One Windows 10/11 host runs the bundle, which contains the server process, the JSON and JSONL persistence tree under data/server/ and the optional Active Directory authentication helper executable. Each student workstation runs the bundle, which contains the client manager, the exam timer GUI (Qt or Tk), the background FFmpeg gdigrab capture process and the CATS web-scraping helper. Both bundles ship with their own offline wheelhouse so the machines can be installed without internet access. All client-server traffic flows through a single switch or wireless access point. The server may consult the institutional CATS portal over HTTPS for credential validation, and may optionally consult an Active Directory

server over LDAP when the AD fallback is enabled. The full deployment topology is shown in Figure 4-5.

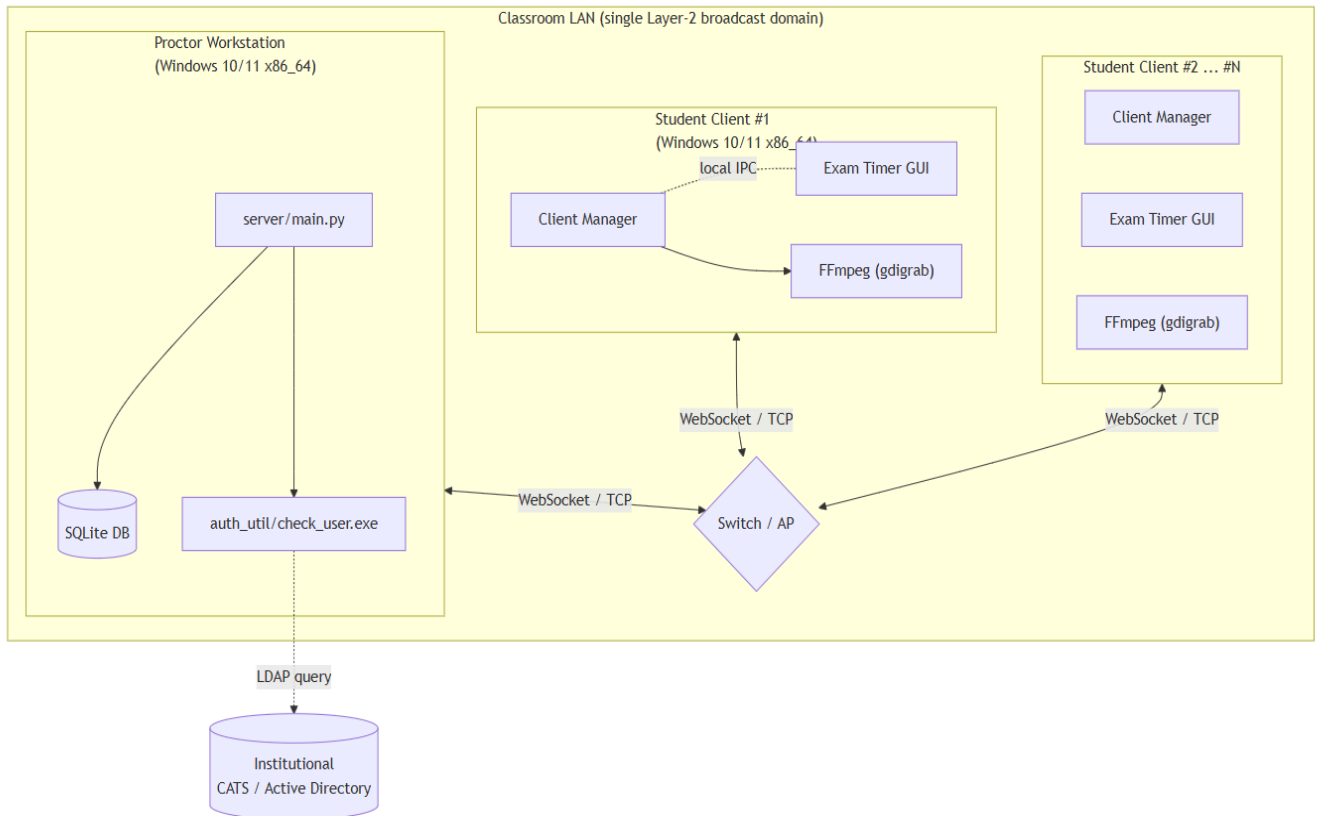
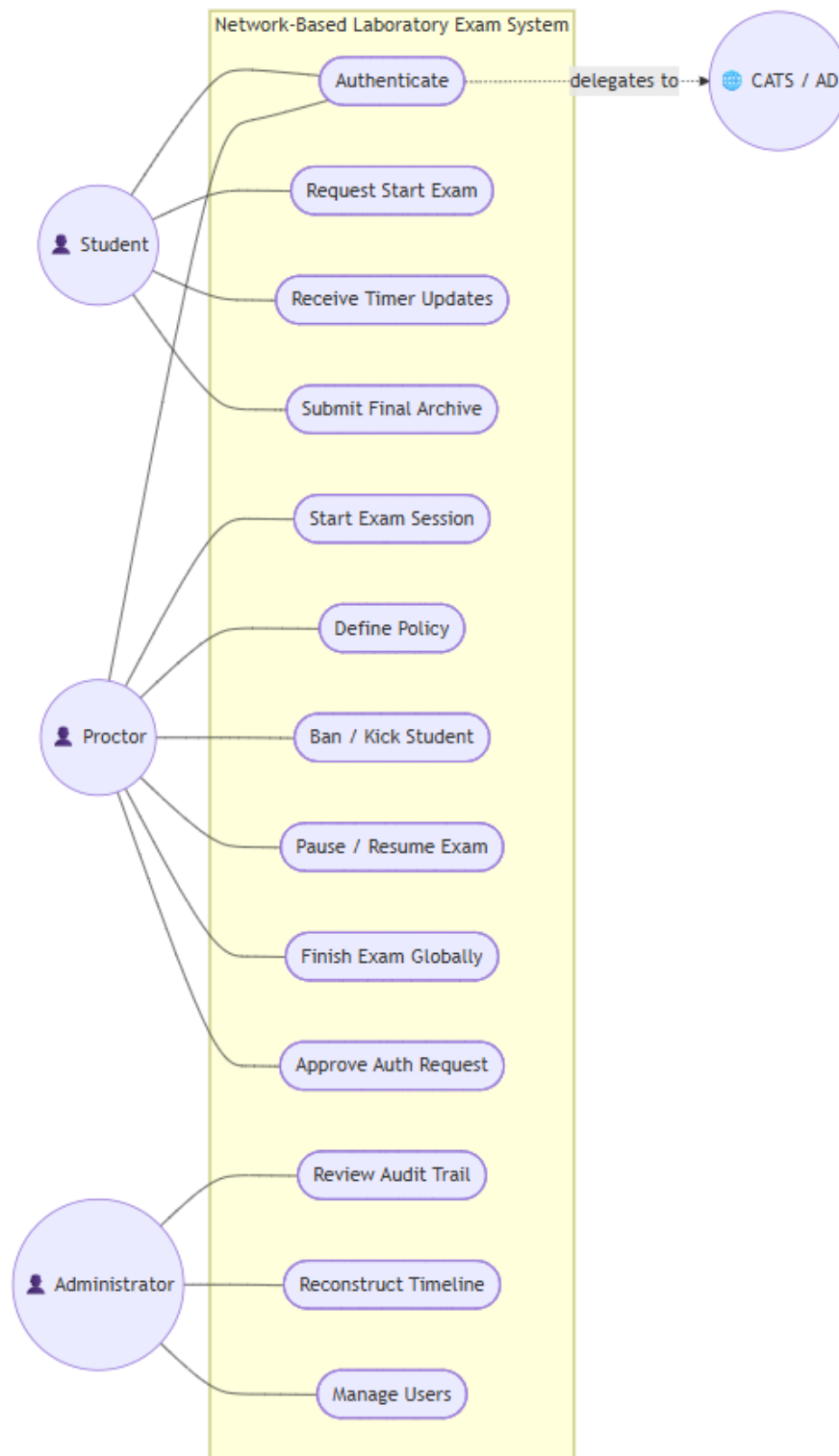


Figure 4-5 — Deployment diagram of the NBLES classroom topology.

4.3 Use Case View

NBLES exposes ten primary use cases, distributed across three actors: the Student, the Proctor (Operator), and the Administrator. The institutional CATS / Active Directory backend is modelled as an external supporting actor. Figure 4-6 summarises the actors and their use cases.



4.3.1 Use Case Scenarios

The following ten scenarios describe the principal interactions with NBLES. Each scenario lists the actor, preconditions, main flow, alternative flows, postconditions, and a short "How to reproduce" tutorial that demonstrates the scenario in a working deployment.

4.3.2 UC-01: Student Authentication

Primary actor: Student

Preconditions:

- The client launcher is running on the student's workstation.
- The classroom LAN is reachable.
- The server is online and accepting connections.

Main flow:

1. The student launches the client.
2. The launcher discovers the server via UDP broadcast.
3. The student enters their institutional username and password.
4. The client encrypts the credentials with the session key.
5. The server forwards the credentials to `auth_util/check_user.exe`.
6. The helper validates against CATS / AD and returns success.
7. The server registers the student in state Connected and responds with the session token.
8. The client opens the Exam Timer window with state "Waiting for exam start."

Alternative flows:

- A1. The helper returns failure → the client displays "Authentication failed" and remains on the login screen.
- A2. CATS is unreachable and the proctor has not enabled fallback → the login fails with a server-side audit entry.

Postconditions:

- The student's session is in state Connected on the server.
- An `auth_success` event is written to the audit trail.

How to reproduce:

1. Start the server: `python -m server.main --id default --gui`
2. Start the client launcher: `python -m client.main`
3. Enter the test CATS account credentials in the login form.
4. Verify that the Exam Timer window appears.

4.3.3 UC-02: Start Exam

Primary actor: Proctor

Preconditions:

- At least one student session is in state Connected.
- The proctor is signed into the operator dashboard.

Main flow:

1. The proctor types /startexam <duration> in the command bar.
2. The server validates the duration and broadcasts an exam_started event to all Connected sessions.
3. Each client transitions its timer state to Running and begins counting down.
4. The server transitions each session to state Running and logs a state transition.

Alternative flows:

- A1. No connected students → server returns "No participants" and the command is a no-op.
- A2. An exam is already running → server rejects the duplicate /startexam.

Postconditions:

- All Connected sessions are in state Running.
- An exam_started audit event is persisted with the duration.

How to reproduce:

1. After at least one student has logged in, run /startexam 60 in the dashboard.
2. Verify that every client's timer begins counting down from 60:00.

4.3.4 UC-03: Monitor Activity and Report Incident

Primary actor: System (Client + Server)

Preconditions:

- A session is in state Running.
- The active policy ruleset contains at least one entry.

Main flow:

1. The client's process and focused-window monitors sample the system state on each tick.
2. A monitor detects a process or window matching a policy rule.
3. The client builds an incident bundle (event metadata + screenshot + process report) in custommodules/incident_bundle.
4. The bundle is uploaded to the server over WebSocket.
5. The server appends the incident to the EVENTS table, updates the dashboard row for that student, and renders an alert card.

Alternative flows:

- A1. The evidence upload fails → the client retries; the dashboard shows "evidence pending".
- A2. The rule severity is "kill" → the client also terminates the offending process and adds the kill outcome to the event.

Postconditions:

- An incident row exists in EVENTS.
- The dashboard row reflects the new incident counter.

How to reproduce:

1. Add notepad.exe to the process blacklist in policy_settings.
2. Start the exam, then open Notepad on a client.
3. Verify the dashboard alert and the EVENTS row.

4.3.5 UC-04: Define or Update Policy

Primary actor: Proctor

Preconditions:

- The proctor is signed into the operator dashboard.

Main flow:

1. The proctor opens Policy Settings.
2. The proctor adds, removes, or modifies entries in the process blacklist, focused-window policy, or whitelist.
3. The proctor clicks Save.
4. The server persists the new policy and broadcasts a policy_updated event.
5. Each client refreshes its in-memory ruleset.

Alternative flows:

- A1. The proctor cancels → no changes are persisted or broadcast.

Postconditions:

- All connected clients use the updated policy ruleset.
- A policy_updated audit event is persisted.

How to reproduce:

1. From the dashboard, open Policy Settings.
2. Add chrome.exe with severity warn.
3. Click Save and verify that an active client logs "policy refreshed".

4.3.6 UC-05: Ban or Kick Student

Primary actor: Proctor

Preconditions:

- A session is in state Running or Paused.
- The proctor is signed into the dashboard.

Main flow:

1. The proctor selects the offending student and issues /ban <login_id> or /kick <login_id> with an optional reason.
2. The server transitions the session to Banned or Kicked.
3. The server sends a close frame to the client with the reason (truncated to 125 UTF-8 bytes).
4. The client displays the reason and disconnects cleanly.

Alternative flows:

- A1. /kick is used with a long Turkish reason → the server truncates on UTF-8 boundary; the client decodes without errors.

Postconditions:

- The session is in state Banned (still tracked) or Kicked (disconnected).
- An instructor_action audit row is persisted.

How to reproduce:

1. From the dashboard, run /ban student01 "Plagiarism suspected".
2. Verify the student row turns red and the client disconnects.

4.3.7 UC-06: Pause and Resume Exam

Primary actor: Proctor

Preconditions:

- At least one session is in state Running.

Main flow:

1. The proctor issues /pause <login_id> or /pause all.
2. The server records remaining_seconds and transitions the session(s) to Paused.
3. The client freezes the timer and displays the pause reason.
4. Later, the proctor issues /resume.
5. The server transitions to Running and the client resumes the timer at the recorded remaining_seconds.

Alternative flows:

- A1. /pause is issued on a non-running session → the command is a no-op.

Postconditions:

- The session is Paused (then Running again after /resume).
- Pause and resume audit events are persisted.

How to reproduce:

1. Run /pause student01 "break".
2. Verify the client timer freezes; then /resume student01.

4.3.8 UC-07: Submit Final Archive

Primary actor: Student

Preconditions:

- The session is in state Running or Paused.
- The student has prepared an archive (.zip / .tar.gz / etc.) containing their answers.

Main flow:

1. The student clicks Finish Exam on the timer.
2. The submission window opens, listing the file preview.
3. The student clicks Choose File and selects the archive.
4. The client validates the archive (size, magic bytes, extension).
5. The student clicks Upload And Finish.
6. The client stops the runtime monitors, exports the process report, hardware snapshot, and focused-window report.
7. The client calls save_replay (FFmpeg merge) to produce a 60-second screen replay.
8. The client packages the bundle and uploads it to the server with checksum verification.
9. The server stores the bundle and acknowledges success.
10. The client displays Submission Uploaded and closes.

Alternative flows:

- A1. The replay save times out → the bundle is uploaded without the replay; a log line records the absence.
- A2. The checksum mismatches on the server → the upload is rejected and the client allows a retry.

Postconditions:

- The session is in state Finished.
- A submission row exists in the database with the bundle path and checksum.

How to reproduce:

1. On a client, click Finish Exam.
2. Choose any small .zip file and click Upload And Finish.
3. Verify the success dialog and the server-side bundle file.

4.3.9 UC-08: Finish Exam Globally

Primary actor: Proctor

Preconditions:

- An exam is in progress (one or more Running sessions).

Main flow:

1. The proctor issues /finishexam.
2. The server iterates every user, skipping those in state Banned.
3. The server sends a finish_exam_global event to each non-banned session.
4. Each client opens its submission window and prompts the student to upload.
5. The server prints the audit line "Skipped N banned user(s)" where N is the number of excluded participants.

Alternative flows:

A1. All participants are banned → N equals the participant count and no submission prompts are sent.

Postconditions:

- Every non-banned session transitions toward Finished as students upload.
- Banned sessions remain in state Banned.
- A finish_exam_global audit event is persisted.

How to reproduce:

1. After running /ban on at least one client, run /finishexam.
2. Verify the audit line and that the banned client receives no submission prompt.

4.3.10 UC-09: Review Audit Trail

Primary actor: Administrator

Preconditions:

- The administrator has a valid RBAC token for the /audit endpoints.
- At least one completed exam exists in the database.

Main flow:

1. The administrator opens the audit review UI (or the HTTP endpoint /audit/sessions).
2. The system lists past sessions filterable by date, user, and verdict.
3. The administrator selects a session and views its events, instructor actions, and state transitions side by side.
4. The administrator records a review verdict via /audit/review.

Alternative flows:

A1. The RBAC token is missing or expired → the endpoint returns HTTP 401.

Postconditions:

- A REVIEWS row is persisted with the verdict and notes.

How to reproduce:

1. curl -H "Authorization: Bearer <token>"

http://localhost:8765/audit/sessions<http://localhost:8765/audit/sessions>

4.3.11 UC-10: Reconstruct Session Timeline

Primary actor: Administrator

Preconditions:

- The administrator has access to /audit/timeline.
- The target session is present in the database.

Main flow:

1. The administrator calls /audit/timeline?session_id=<id>.
2. The server runs reconstruct_timeline(), joining EVENTS, INSTRUCTOR_ACTIONS, and STATE_TRANSITIONS in chronological order.
3. The system returns a JSON timeline including every operator command, every incident, and every state change with their timestamps and originating user.

Alternative flows:

A1. The session id does not exist → HTTP 404.

Postconditions:

- The caller has a deterministic reconstruction of the session usable in academic integrity hearings.

How to reproduce:

1. Pick a session_id from /audit/sessions and call /audit/timeline?session_id=<id>.

5 Design and Implementation

The implementation follows the architectural decomposition presented in Section 4. The server is an asynchronous Python application built on aiohttp and the websockets library; it exposes a single WebSocket endpoint for client traffic, a small HTTP surface for the audit and projector endpoints, and a local IPC channel for the dashboard. The client is split into a manager process and a timer GUI, communicating over a loopback IPC channel hardened with a per-process token. Activity is observed by five independent monitor packages under client/client/custommodules (focused_window_monitor, hardware_monitor, idle_monitor, process_monitor, replay_recorder), each with a thin and a platform-specific adapter. Outbound events are buffered by client/client/outbound_buffer.py so transient WebSocket disconnects never lose data. The submission flow shown in Figure 5-1 is the most involved end-to-end path: it stops the runtime monitors, captures the closing snapshots, drives an FFmpeg merge for the 60-second replay, builds the submission bundle with per-file SHA-256 checksums, and uploads it to the server under archive-level checksum verification. Cryptographic operations are centralised in common/security.py; the dashboard and policy settings UI live in server/server/ui/dashboard_qt.py, and with the reusable component set in common_ui/widgets.py and common_ui/styles.py.

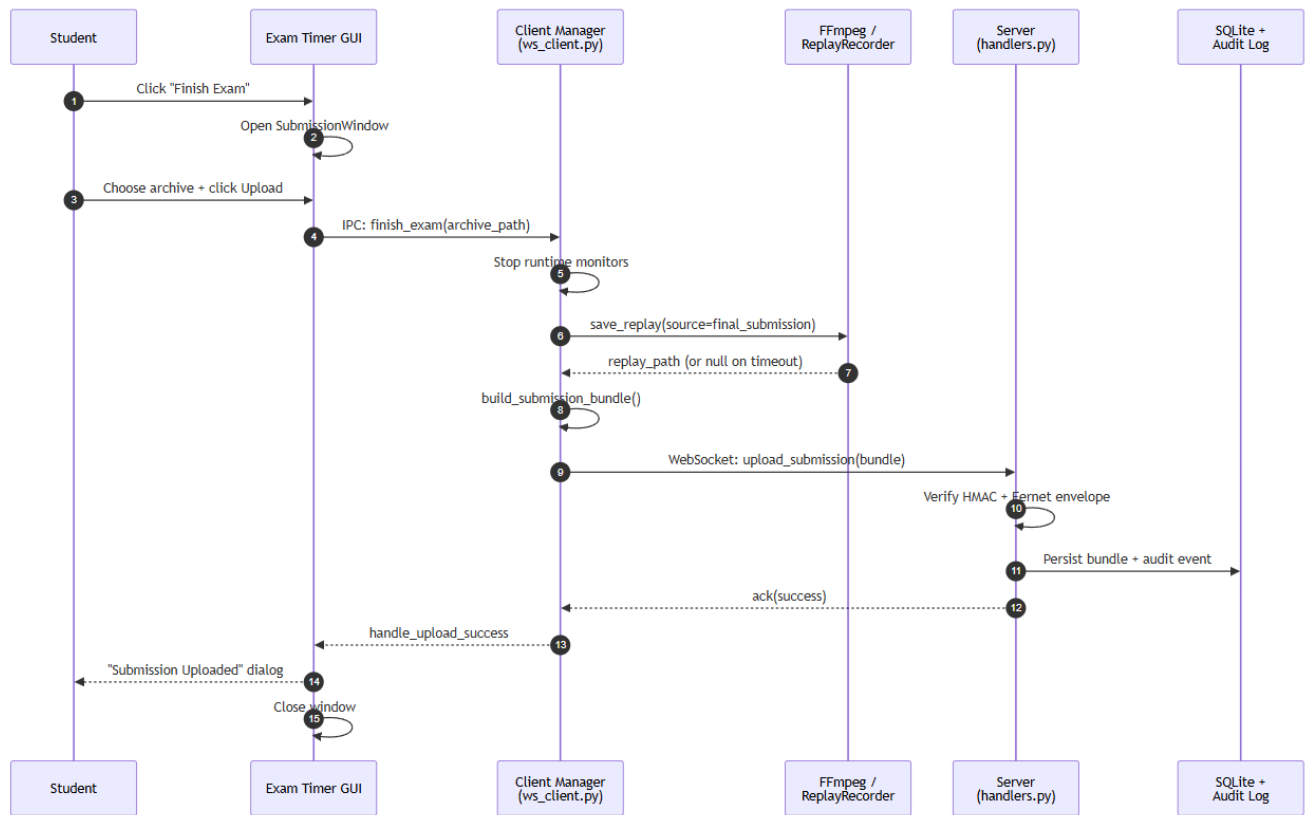


Figure 5-1 — Sequence diagram of the finish-exam and submission upload flow.

6 Other Supporting Information

This section consolidates supporting information that does not fit elsewhere in the SRS.

6.1.1 Problem Statement Recap

Traditional in-person lab examinations rely on the proctor walking between rows of workstations to observe student screens. This is labour-intensive, inconsistent across proctors, and leaves no auditable trace if a violation is later contested. NBLES replaces the manual loop with a single dashboard that surfaces every student's state, every detected violation, and every operator command in real time, while persisting the same information for post-exam review.

6.1.2 Sample I/O Formats

WebSocket payloads use the protocol envelope defined in `common/protocol.py`. A representative incident payload, after encryption is stripped, is:

```
{ "msg": "incident.report", "session_id": "...", "incident": { "id": "...", "type":  
"focused_window_match", "window_title": "ChatGPT — Google Chrome", "severity":  
"auto_pause", "ts": "2026-05-19T10:31:45Z" }, "evidence": { "screenshot": "<base64>",  
"process_report": [...] } }
```

6.1.3 Packaging Notes

The full distribution consists of the source tree, a `requirements.txt` for the Python dependencies, a pre-built `auth_util/check_user.exe` for the CATS / AD helper, and the FFmpeg binaries. For deployment outside the institutional network the CATS configuration must be substituted; see Section 7.5 for the fallback workflow.

7 Deployment and Operation Guide

This section explains how to deploy NBLES on a fresh Windows 10 or Windows 11 machine — first the server, then any number of clients. The May_12 layout ships two independent bundles, and , each with its own and its own offline-packages folder. All commands below are written for Windows PowerShell or cmd.

7.1.1 Hardware and Software Prerequisites

Server: Windows 10 or Windows 11 (x86_64), 4 GB RAM minimum, 5 GB free disk, wired Ethernet to the classroom switch. Client: same operating system, 2 GB RAM minimum, 500 MB free disk (for the rolling replay cache), and administrator privileges only for the initial Python installation.

7.1.2 Installing Python and Third-Party Libraries

Download and install Python 3.13 from python.org. During the installer, tick "Add Python to PATH" and "Install launcher for all users". Verify with: `py -3.13 --version`. The bundle setup script will refuse to proceed on any other interpreter version. Then, from the chosen bundle folder, run `python --offline` to install the Python dependencies from the local wheelhouse without internet access, or `python` for an online install. The setup script installs dependencies with `pip install --user`; no virtual environment is created and no system-wide install is performed.

7.1.3 Installing FFmpeg and Adding It to PATH

The client bundle ships FFmpeg under The runtime looks up FFmpeg in this order: the EXAM_FFMPEG_PATH environment variable, then the bundled offline-packages/ffmpeg/bin folder, then the system PATH. If the client logs "[RECORDER] ERROR: FFmpeg not found", set EXAM_FFMPEG_PATH to the full path of ffmpeg.exe or copy the binaries into offline-packages/ffmpeg/bin.

7.1.4 Building the C++ Authentication Helper

The Active Directory fallback helper, `auth_util/check_user.cpp`, is optional in the May_12 layout: the main authentication route is CATS web scraping through `client/school_service.py`. The helper only needs to be compiled when the operator sets `ad_domain` in `server/auth_config.json` and intends to use the AD fallback. To build it, install the Visual Studio Build Tools or MinGW-w64 and run: `g++ -std=c++17 -O2 auth_util/check_user.cpp -o auth_util/check_user.exe -lldap32`.

7.1.5 Network Configuration

Ensure the server machine and all client machines are on the same Layer-2 subnet. Open the following ports in the Windows Firewall on the server: TCP 8765 (WebSocket and HTTP), UDP 5354 (UDP discovery beacon). The server console prints the bound IP and port at startup; if UDP discovery fails on a particular network (common on overlay adapters), the client can be launched with an explicit `--host` and `--port` argument to bypass discovery.

7.1.6 Persistence Initialisation

On first startup the server creates the `data/server/` tree if it does not exist and writes the default persistence files: `server_users.json` (empty), `incidents.jsonl` (empty), `exam_policy.json` (default policy bundle), `incident_rules.json` (default rule set), `process_blacklist.txt` (empty), and `process_definitions.json` (default definitions). No external database server is required. On every subsequent start the files are normalised: missing fields are added with defaults, legacy field names are translated, and any partially written last line in the append-only logs is recovered.

7.1.7 Environment Variables and Configuration Files

The following configuration files are honoured at runtime: `server/auth_config.json` (`auth_secret` and `ad_domain`), `server/allowed_users.json` (admission whitelist), `server/data/server/exam_policy.json` (default exam policy), `incident_rules.json` (incident rule set), `process_blacklist.txt` (line-delimited process names), `process_definitions.json` (process metadata). The `EXAM_FFMPEG_PATH` environment variable overrides the FFmpeg lookup order described above.

7.1.8 Running the Server

From , double-click `run_server_qt.bat` for the Qt dashboard, `run_server_tk.bat` for the Tkinter dashboard, or `run_server_cli.bat --help` for the headless command-line mode. The batch scripts prefer `py -3.13` and fall back to `python` only when it resolves to Python 3.13. The console prints the bound IP and port; the operator dashboard opens automatically in Qt and Tk modes.

7.1.9 Running the Client

From on each student machine, double-click `run_client_qt.bat` for the Qt timer GUI, `run_client_tk.bat` for the Tkinter timer GUI, or `run_client_cli.bat --help` for headless operation. The client manager discovers the server through UDP broadcast and presents the login form. The student enters their CATS credentials; on successful authentication the exam timer window appears.

7.1.10 Default Login Credentials

Production credentials are supplied by the institutional CATS portal; no local password database is maintained by NBLES. For offline testing, the operator may temporarily disable CATS authentication through `/disablecatsauth 60` and approve individual login requests through `/approveauth <login_id>`. The default TTL is sixty seconds and the server clamps the maximum. The fallback is intended for development; production deployments should always rely on the live CATS portal.

7.1.11 Verifying a Working Deployment

(1) The server console prints "[SERVER] Listening on ...". (2) A second attempt to start the server prints "[ERROR] Duplicate server detected" and exits. (3) At least one client connects and appears in the dashboard. (4) The bundle's unit tests pass: from run python -m unittest discover -s tests, and from run python -m unittest discover -s tests. (5) The operator commands /startexam, /pause, /resume, /ban, /kick and /finisexam each yield the expected console audit line and dashboard update.

7.1.12 Troubleshooting

"FFmpeg not found" — set EXAM_FFMPEG_PATH or repopulate offline-packages/ffmpeg/bin.

"Duplicate server detected" — another instance is already listening; stop it before retrying.

"Connection refused" from the client — verify the firewall rules above and that the server is on the same subnet, or run the client with explicit --host and --port. "CATS unavailable" — use

/disablecatsauth and /approveauth as described in 7.10, then restore CATS validation once the portal is reachable again. "Python version mismatch" — will print the installed version and exit; install Python 3.13 and retry.

This section is the official setup guide for installing the System on a fresh Windows 10 or Windows 11 machine. The same seven-step procedure applies to both the server host and every student client host. The instructions are also distributed as a stand-alone setup card so an operator can carry them to a machine without opening this document.

Step 1 — Install Python 3.13

Download and install Python 3.13 from python.org. During installation the following installer options must be enabled.

<https://www.python.org/downloads/>

During installation, make sure these options are enabled:

Add Python to PATH

Install launcher for all users

After installation finishes, click:

Disable path length limit

Step 2 — Open Command Prompt as Administrator

Open the Start Menu and search for:

cmd

Right-click Command Prompt and choose:

Run as administrator

Step 3 — Install FFmpeg

In the administrator command prompt, run:

winget install ffmpeg

Step 4 — Go to the Project Folder

Change directory into the extracted project folder. The exact path depends on where the bundle was unpacked; the example below uses a placeholder.

```
cd C:\Path\To\Project
```

Step 5 — Install Python Dependencies

Run the following commands in order:

```
python -m pip install --upgrade pip
python -m pip install aiohttp psutil cryptography PySide6 requests==2.33.1 beautifulsoup4
```

Step 6 — Run Setup

Run:

```
python setup.py
```

If setup.py fails, run:

```
python -m pip install -r requirements.txt
```

If there is no requirements.txt, run this again:

```
python -m pip install aiohttp psutil cryptography PySide6 requests==2.33.1 beautifulsoup4
```

Step 7 — Check Installation

Run:

```
python --version
pip --version
ffmpeg -version
where python
where pip
where ffmpeg
```

Expected:

```
Python 3.13.x
pip works
ffmpeg works
all where commands return a path
```

8 Design Decisions — Before and After

This section records the principal design decisions taken at the start of the NBLES project and how they evolved into the final May_12 implementation. The entries are limited to the changes that altered the system's architecture, deployment model or security posture; routine refactors are not listed. Each entry contrasts the initial "before" state with the final "after" state and briefly explains the motivation for the change.

Project layout

BEFORE The first working iteration of the project lived as a flat folder of approximately eighteen Python modules together with a Jupyter notebook (server.ipynb) in which the server behaviour was first prototyped. Client, server and shared code coexisted in the same directory; the deliverable was a pair of demo scripts.

AFTER The system is now organised into two physically independent deployment bundles, and , each carrying its own, requirements.txt and offline-packages folder. Shared code is duplicated into a common/ package inside both bundles so neither contains the other's runtime. A VALIDATION.md inside May_12 enforces this boundary.

Reason for change: The single-folder layout coupled the operator and student code paths in a way that made deployment to two different machines fragile, and the notebook prevented automated testing. The split bundles let the operator and the students install only what they need, on isolated machines, without exposing one side's code to the other.

Distribution and offline install

BEFORE Installation assumed internet access on every machine and expected the operator to install FFmpeg, Python, the Python dependencies and the optional native helpers by hand. The exact dependency versions were not pinned.

AFTER Each bundle ships an offline-packages folder containing a pre-built pip wheelhouse, the FFmpeg binaries and a manifest.sha256 integrity file. The bundle's installs dependencies with pip install --user --no-index --find-links offline-packages/wheelhouse, so an internet-isolated lab machine can be set up start to finish without contacting any remote server.

Reason for change: Lab machines used during exams are typically on a network without unrestricted internet access, and on-the-day installation problems are the single largest deployment risk. The offline bundle removes the network and the operator from the critical path.

Python interpreter requirement

BEFORE No specific interpreter version was enforced; the codebase was developed against whatever version each contributor happened to use, and runtime errors surfaced occasionally on older interpreters.

AFTER The bundle checks TARGET_PYTHON = (3, 13) and refuses to proceed on any other version. The launcher batch scripts (run_*_qt.bat, run_*_tk.bat, run_*_cli.bat) prefer py -3.13 and fall back to python only when it resolves to Python 3.13.

Reason for change: Pinning the interpreter to Python 3.13 lets us rely on modern syntax (PEP 604 unions, structural pattern matching, improved tomllib) and removes a class of subtle compatibility bugs that appeared on mixed-version deployments.

User interface stack

BEFORE The early prototype had no real UI; behaviour was driven from a notebook and a pair of demo scripts. There was no dedicated dashboard for the operator and no timer window for the student.

AFTER The operator runs a full PySide6 dashboard with feature parity in a Tkinter back-end; both back-ends share the reusable widget set in common_ui/. The student runs a minimal timer that surfaces only the exam state, the remaining time and the submission window. A third command-line back-end is provided for headless operation.

Reason for change: Different machines impose different UI constraints: operator stations need a rich dashboard, student lab machines need a minimal timer, and projection machines or scripted test environments need a headless mode. Providing three coordinated back-ends covers all three deployments from a single codebase.

Configuration model

BEFORE Server IP, student identifier, exam identifier, shared secret and policy lists were declared as module-level constants inside the source files. Changing any of them required editing the source and redistributing the bundle.

AFTER Configuration is now held in external JSON files: `server/auth_config.json` (`auth_secret`, `ad_domain`), `server/allowed_users.json` (admission whitelist), `server/data/server/exam_policy.json` (default exam policy), `incident_rules.json`, `process_blacklist.txt` and `process_definitions.json`. The dashboard exposes a Policy Settings dialog that edits these files at runtime and broadcasts the change to every connected client.

Reason for change: Hard-coded constants meant a separate build per exam, and any last-minute change risked breaking the runtime. External JSON lets the operator update credentials, whitelists and rules without restarting the server and without recompiling anything.

Authentication

BEFORE The first prototype carried its own HMAC-signed credential exchange and a separate role-based check for the instructor. CATS integration existed but was a side-by-side route rather than the primary one, and the shared secret was hard-coded in.

AFTER Authentication is now built around a single path: `client/school_service.py` scrapes the live CATS portal, verifies that the scraped student identifier matches the one entered locally, and forwards the result to the server, which additionally enforces the admission whitelist in `server/allowed_users.json`. The native Active Directory helper `auth_util/check_user.cpp` remains in the tree as an optional fallback controlled by `ad_domain` in `auth_config.json`, and a runtime-only `/disablecatsauth/approveauth` workflow covers the case when the CATS service is unreachable.

Reason for change: Multiple parallel authentication paths produced inconsistent failure modes and made it difficult to reason about who was admitted to the exam. Centralising on CATS with a defined fallback gave us a single auditable entry point, and externalising the whitelist let the operator manage admission without touching the source.

Activity monitoring

BEFORE All monitoring code lived in a single module: focused window, running processes and idle detection were handled in one class with platform branches inside individual methods. Screen capture and hardware snapshots did not exist.

AFTER Monitoring is now split into five independent packages under `client/client/custommodules`: `focused_window_monitor`, `process_monitor`, `idle_monitor`, `hardware_monitor` and `replay_recorder`. Each package isolates its platform-specific code in an adapter behind a thin public interface, which lets each monitor be exercised by its own unit test.

Reason for change: A single module had become difficult to extend and to test, and adding a new monitor required edits scattered across the file. The per-monitor package layout makes each addition self-contained and lets a new platform (future Linux or macOS support) be added by writing one adapter file rather than refactoring the whole module.

Session state model

BEFORE Server-side session state was tracked with two flags: a string "in_progress" or "completed", and a separate banned boolean. Edge cases such as banned-during-finish or paused-during-reconnect produced inconsistent results depending on which flag was read first.

AFTER Sessions move through an eight-state machine declared in `server/server/session_state.py` (WAITING, RUNNING, ADMIN_PAUSED, DISCONNECTED_PAUSED, VIOLATION_PAUSED, AWAITING_SUBMISSION, SUBMITTED, FINISHED, with explicit BANNED and KICKED terminal nodes). Every transition is audited and every command checks the current state before acting.

Reason for change: The flag-pair model could not express the difference between an administrative pause, a network-loss pause and a violation-driven pause, all of which need different handling on resume. A named state machine gives both the operator and the auditor a clear vocabulary and removes an entire class of "who was active when" bugs.

Submission and evidence collection

BEFORE The original prototype had no submission workflow at all: an exam ended when its timer expired, and the only record left behind was the rows in `monitoring_events` and `violations`.

AFTER At the end of the exam the client builds a submission bundle containing the student archive, a runtime process report, a hardware snapshot, a focused-window report and a 60-second screen replay captured by FFmpeg `gdigrab`. Each file is hashed with SHA-256 and the bundle carries an archive-level `archive_sha256`; the server endpoint rejects a mismatched archive and accepts retries from the student without re-prompting for the file.

Reason for change: Academic integrity hearings need a defensible record of what was on the machine at the moment of submission, not only what the monitors observed during the exam. The bundle with checksums gives the auditor an artefact that is both complete and tamper-evident.

Persistence and audit trail

BEFORE Persistence was a single SQLite file with five tables and no formal audit trail beyond the `audit_log` table; operator commands and state transitions were not consistently recorded, and there was no documented way to reconstruct what had happened in a session after the fact.

AFTER The persistence layer was rewritten as a small set of JSON and JSONL files under `data/server/` and `data/client/{uuid}/`. Editable files (`server_users.json`, `exam_policy.json`, `incident_rules.json`, `process_definitions.json`) are rewritten atomically through a temp-file-then-rename pattern; append-only logs (`incidents.jsonl`, the durable outbound buffer packets.jsonl) tolerate a single truncated last line. The schema is created on first run and normalised on every subsequent start so a fresh checkout boots cleanly.

Reason for change: The original SQLite-backed prototype tightly coupled the runtime to a single binary file format that was opaque to operators and hard to inspect after the fact. Plain JSON and JSONL files can be opened with any text editor, version-controlled when needed, and verified row by row during incident review without specialised tools.

Classroom projector view

BEFORE There was no dedicated classroom display; the operator's screen was the only place to see overall exam state.

AFTER The server now exposes a /projector HTTP page served from server/server/static/projector/ (index.html, projector.css, projector.js) that streams updates through /projector/events using Server-Sent Events. Any HTML5 browser on the classroom network can mirror the projector view without requiring a desktop UI on the projection machine.

Reason for change: Mirroring the operator's dashboard for the classroom was unsuitable: it shows operational detail that students should not see. A separate, read-only projector page lets the room see only the aggregate, public information (time remaining, current phase, public notices).

Before / After: Visual Comparison of the Integrated Build

This section presents a visual comparison between the early prototype iterations of the System and the final integrated build delivered at the end of the project. Each subsection follows the same structure: a BEFORE narrative describing the prior behaviour, the corresponding screenshot evidence where applicable, an AFTER narrative describing the final behaviour, the corresponding screenshot, and a short reason for the change. Author names are intentionally not used; the focus is on what the System used to be and what it has become.

1. Wire Protocol and Secure Communication

BEFORE

The earliest iteration of the System transmitted student-side monitoring data over a plain JSON WebSocket connection. Messages carried only the event payload, with no checksum, no signature and no replay protection. Any process able to send a well-formed JSON frame to the server was effectively trusted. Sensitive operator events such as pause or termination were sent in clear text and could in principle be replayed by an attacker who captured a single recording of the network traffic.

AFTER

Every message on the wire now follows a three-field envelope consisting of an event name, a data payload and a SHA-256 checksum computed over the canonical JSON serialisation. Sensitive events are additionally wrapped by a session security context that signs the envelope with HMAC, includes a per-message nonce and timestamp, optionally encrypts the payload with Fernet and enforces a server-side replay window. The result is that even an attacker with full network capture cannot replay an old pause command, and any byte-level tampering is rejected before the handler sees the message.

Reason for change: Plain JSON over WebSocket was easy to develop against but did not satisfy the integrity and replay-protection requirements of an academic-integrity tool. Layering the security context on top of the envelope kept the development loop fast while making the wire format defensible.

2. Monitoring Pipeline and Network Reliability

BEFORE

The monitoring layer originally ran as a standalone command-line loop on the student machine. Each tick collected the active window title, the process list and the idle time and printed the result to the terminal as a line of log text. Violations were printed to the same console as ALARM lines, and the JSON payload was visible on screen as raw text. When the WebSocket dropped, the loop continued to log but events accumulated in memory only; on a long outage they were lost when the process exited.

```
=====
MONITOR LOOP - live mode
Ctrl+C to stop
=====
[AUTH] Credentials prepared for '2300006437'
[NET] Session ID : 41dfa8d3-93c9-45b5-a7f3-7ac489f70795
[NET] Waiting room: DISABLED (test mode)
[NET] Connected to ws://127.0.0.1:8765
[NET] ✓ Authenticated: Hoş geldin Engin Erkurt. Öğitmen sınavı başlatana kadar lütfen bekleyiniz.
[NET] ▶ Waiting room skipped (SKIP_WAITING_ROOM=True).
[MONITOR] Loop started.
[HB] Engin Erkurt | window='Komut İstemi - python monitor_loop.py' | idle=2s | flags=[BANNED:whatsapp.root]
[NET] ✓ Sent [encrypted] seq=1 | violation=True | flags=[BANNED:whatsapp.root]
[HB] Engin Erkurt | window='Komut İstemi - python monitor_loop.py' | idle=7s | flags=[BANNED:whatsapp.root]
[NET] ✓ Sent [encrypted] seq=2 | violation=True | flags=[BANNED:whatsapp.root]
[HB] Engin Erkurt | window='Komut İstemi - python monitor_loop.py' | idle=0s | flags=[BANNED:whatsapp.root]
[NET] ✓ Sent [encrypted] seq=3 | violation=True | flags=[BANNED:whatsapp.root]
[HB] Engin Erkurt | window='Komut İstemi - python monitor_loop.py' | idle=0s | flags=[BANNED:whatsapp.root]
[NET] ✓ Sent [encrypted] seq=4 | violation=True | flags=[BANNED:whatsapp.root]
[HB] Engin Erkurt | window='Ekran Alıntısı Aracı Katmanı' | idle=0s | flags=[FOCUS_LOST, BANNED:whatsapp.root]
[NET] ✓ Sent [encrypted] seq=5 | violation=True | flags=[FOCUS_LOST, BANNED:whatsapp.root]
```

Console output of the standalone monitor loop, with encrypted heartbeats and violation flags printed line by line.

AFTER

The monitoring layer now emits structured events through a reliable outbound buffer with monotonically increasing sequence numbers. Each event is encrypted on the wire, persisted to a durable on-disk buffer until the server acknowledges receipt, and deduplicated by sequence identity on reconnect so the server never sees the same event twice. The operator no longer reads raw logs from a terminal: the same events surface in the integrated dashboard as structured rows in the live log and as updates on the per-student detail panel.

```
[{"timestamp": "2026-04-20T16:22:51.686085", "event_type": "student_authenticated", "details": {"student_id": "2300006437", "name": "Engin Erkurt"}}]
[{"timestamp": "2026-04-20T16:22:52.500084", "event_type": "violation_alert", "details": {"student_id": "2300006437", "score": 0, "level": "D\u00dc\u015e\u00dck", "type": "BANNED:whatsapp.root", "window": "Komut \u0130stemi - python monitor_loop.py", "seq": 5, "buffered": false, "queued_at": "2026-04-20T16:22:52.489"}}]
[{"timestamp": "2026-04-20T16:22:57.503965", "event_type": "violation_alert", "details": {"student_id": "2300006437", "score": 0, "level": "D\u00dc\u015e\u00dck", "type": "BANNED:whatsapp.root", "window": "Komut \u0130stemi - python monitor_loop.py", "seq": 6, "buffered": false, "queued_at": "2026-04-20T16:22:57.501"}}]
[{"timestamp": "2026-04-20T16:22:58.472660", "event_type": "violation_alert", "details": {"student_id": "2300006437", "score": 0, "level": "D\u00dc\u015e\u00dck", "type": "BANNED:whatsapp.root", "window": "Komut \u0130stemi - python monitor_loop.py", "seq": 1, "buffered": true, "queued_at": "2026-04-20T16:22:32.465"}}]
[{"timestamp": "2026-04-20T16:22:58.486364", "event_type": "violation_alert", "details": {"student_id": "2300006437", "score": 0, "level": "D\u00dc\u015e\u00dck", "type": "BANNED:whatsapp.root", "window": "Komut \u0130stemi - python monitor_loop.py", "seq": 2, "buffered": true, "queued_at": "2026-04-20T16:22:37.471"}}]
[{"timestamp": "2026-04-20T16:22:58.496000", "event_type": "violation_alert", "details": {"student_id": "2300006437", "score": 0, "level": "D\u00dc\u015e\u00dck", "type": "FOCUS_LOST", "window": "Komut \u0130stemi", "seq": 3, "buffered": true, "queued_at": "2026-04-20T16:22:42.477"}}]
[{"timestamp": "2026-04-20T16:22:58.507075", "event_type": "violation_alert", "details": {"student_id": "2300006437", "score": 0, "level": "D\u00dc\u015e\u00dck", "type": "BANNED:whatsapp.root", "window": "Komut \u0130stemi - python server.py", "seq": 4, "buffered": true, "queued_at": "2026-04-20T16:22:47.483"}}]
```

Structured event payloads written to the audit stream with sequence numbers, buffered flags and queued-at timestamps.

Reason for change: A terminal-only monitor was unusable for a proctor who needs to supervise tens of students simultaneously. Wrapping the same data in a sequence-numbered, encrypted, buffered transport produced a pipeline that survives transient network loss and feeds the dashboard with the same fidelity as the source.

The screenshot below shows the durable offline buffer in action: when the server became unreachable, events were queued and persisted locally, and on reconnect they were flushed in order without duplication.

```
[MONITOR] Offline - buffered. Queue: 1
[HB] Engin Erkurt | window='Komut İstemi - python monitor_loop.py' | idle=8s | flags=[BANNED:whatsapp.root]
[OFFLINE] Not connected - seq=2 buffered. Buffer: 2. Reconnecting...
[MONITOR] Offline - buffered. Queue: 2
[OFFLINE] Server not reachable at ws://127.0.0.1:8765
[HB] Engin Erkurt | window='Komut İstemi' | idle=0s | flags=[FOCUS_LOST, BANNED:whatsapp.root]
[OFFLINE] Not connected - seq=3 buffered. Buffer: 3. Reconnecting...
[MONITOR] Offline - buffered. Queue: 3
[OFFLINE] Server not reachable at ws://127.0.0.1:8765
[HB] Engin Erkurt | window='Komut İstemi - python server.py' | idle=0s | flags=[BANNED:whatsapp.root]
[OFFLINE] Not connected - seq=4 buffered. Buffer: 4. Reconnecting...
[MONITOR] Offline - buffered. Queue: 4
[OFFLINE] Server not reachable at ws://127.0.0.1:8765
[NET] Connected to ws://127.0.0.1:8765
[NET] ✓ Authenticated: Hoş geldin Engin Erkurt. Eğitimci sınavı başlatana kadar lütfen bekleyiniz.
[NET] ▶ Waiting room skipped (SKIP_WAITING_ROOM=True).
[HB] Engin Erkurt | window='Komut İstemi - python monitor_loop.py' | idle=0s | flags=[BANNED:whatsapp.root]
[NET] ✓ Sent [encrypted] seq=5 | violation=True | flags=[BANNED:whatsapp.root]
[HB] Engin Erkurt | window='Komut İstemi - python monitor_loop.py' | idle=4s | flags=[BANNED:whatsapp.root]
[NET] ✓ Sent [encrypted] seq=6 | violation=True | flags=[BANNED:whatsapp.root]
[NET] Flushing 4 buffered packets...
[NET] Flushing 4 buffered packet(s)...
[NET] ✓ Flushed seq=1
[NET] ✓ Flushed seq=2
[NET] ✓ Flushed seq=3
[NET] ✓ Flushed seq=4
[NET] WebSocket closed.
[NET] Background loop stopped.
[MONITOR] Loop stopped.
Exited.
```

Offline-mode buffer queueing four events, then flushing them in sequence on reconnect.

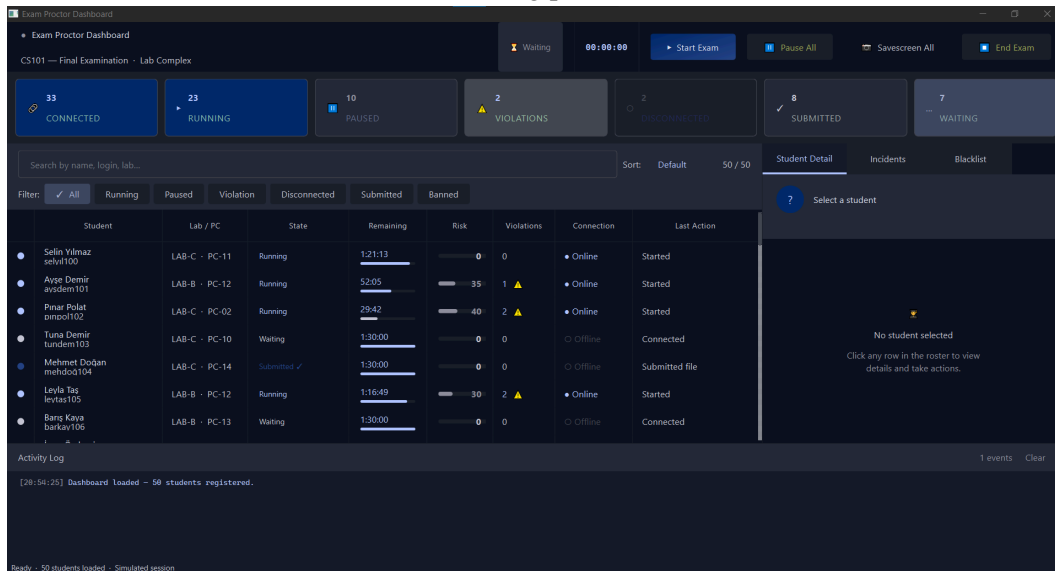
3. Operator Dashboard: Live Roster and Filtering**BEFORE**

An early dashboard iteration presented a styled summary header with coloured cards for the connection, running, paused, violation, disconnected, submitted and waiting counts, followed by a filterable table of students with state, remaining time, risk, violations, connection and last action columns. The presentation

Software Requirements Specification for Exam Software Application

Page 40

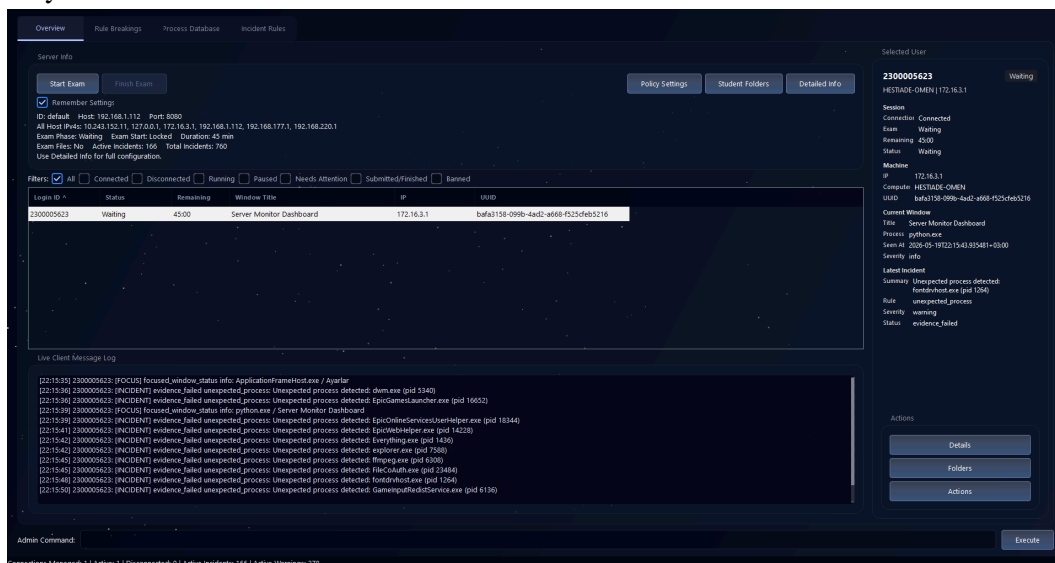
was clean but the dashboard was a thin display layer: operator actions were not yet wired through a centralised command surface, and there was no live-log panel to observe individual incidents as they arrived.



Early dashboard with stylised summary cards, filter chips and student roster.

AFTER

The integrated dashboard organises the same data under four named tabs (Overview, Rule Breakings, Process Database, Incident Rules). The Overview tab combines a compact server-info panel with the student roster, a live client message log and an always-visible admin command bar at the bottom of the window. A right-hand panel shows the selected user's session, machine, current window and latest incident in a single read. Filters now operate against the same data the operator commands act on, so the visible roster always reflects the true state of the System.



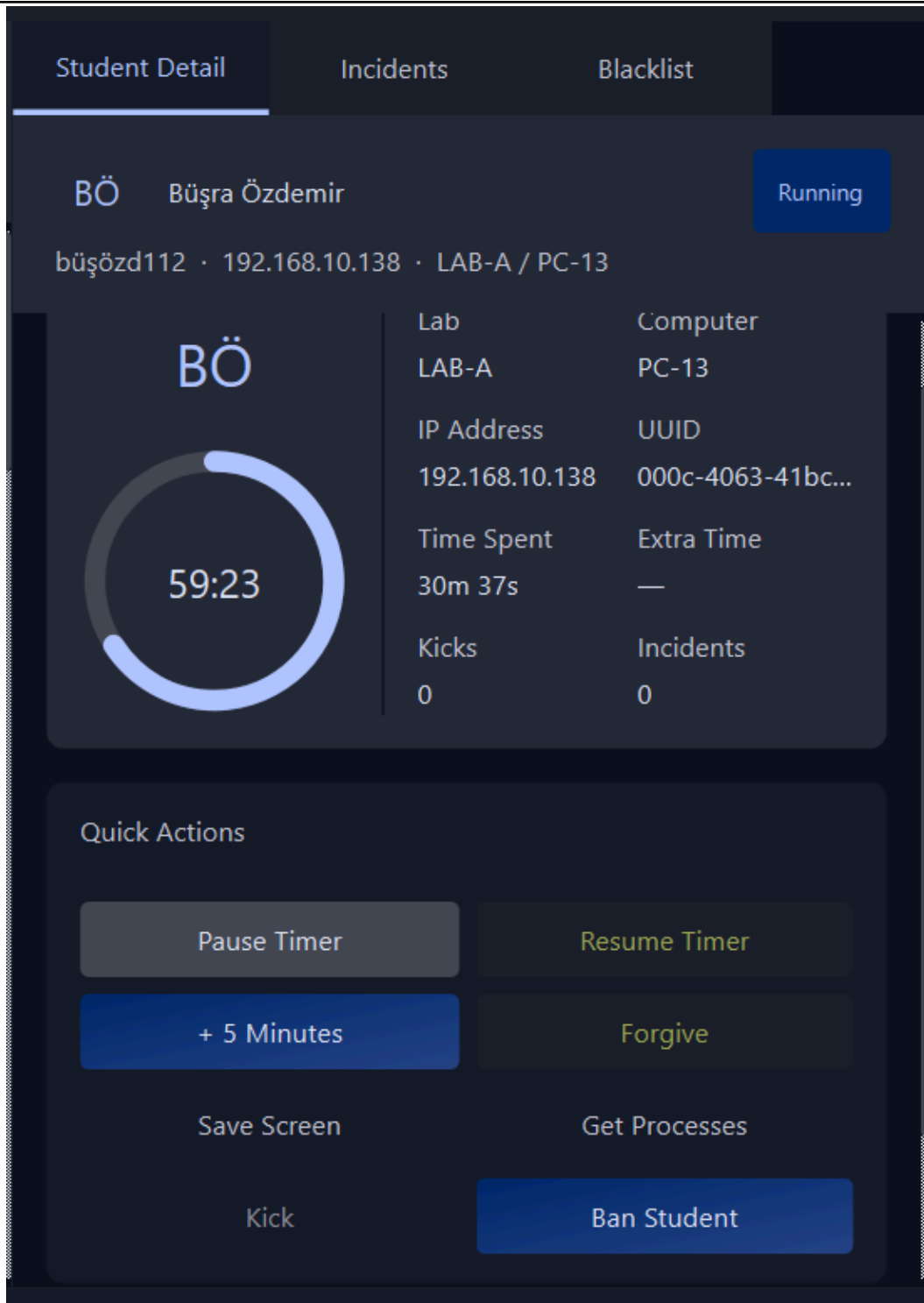
Integrated dashboard with tabbed navigation, live client message log, admin command bar and the Selected User detail panel.

Reason for change: The early dashboard was visually pleasing but did not expose the operator's full command surface. The integrated version trades some visual polish for direct operational reach: every command issued from the right-hand actions, the admin command bar or a keyboard shortcut lands on the same dispatcher as the typed CLI command.

4. Per-Student Detail Panel

BEFORE

The first per-student view was a polished card containing the student's initials, full name, a circular countdown timer, a compact lab/computer/IP/UUID block, time-spent and extra-time fields, kick and incident counters and a Quick Actions area with labelled buttons for Pause Timer, Resume Timer, plus five minutes, Forgive, Save Screen, Get Processes, Kick and Ban Student. The view was elegant but mostly read-only: actions were buttons without a unified command channel behind them.

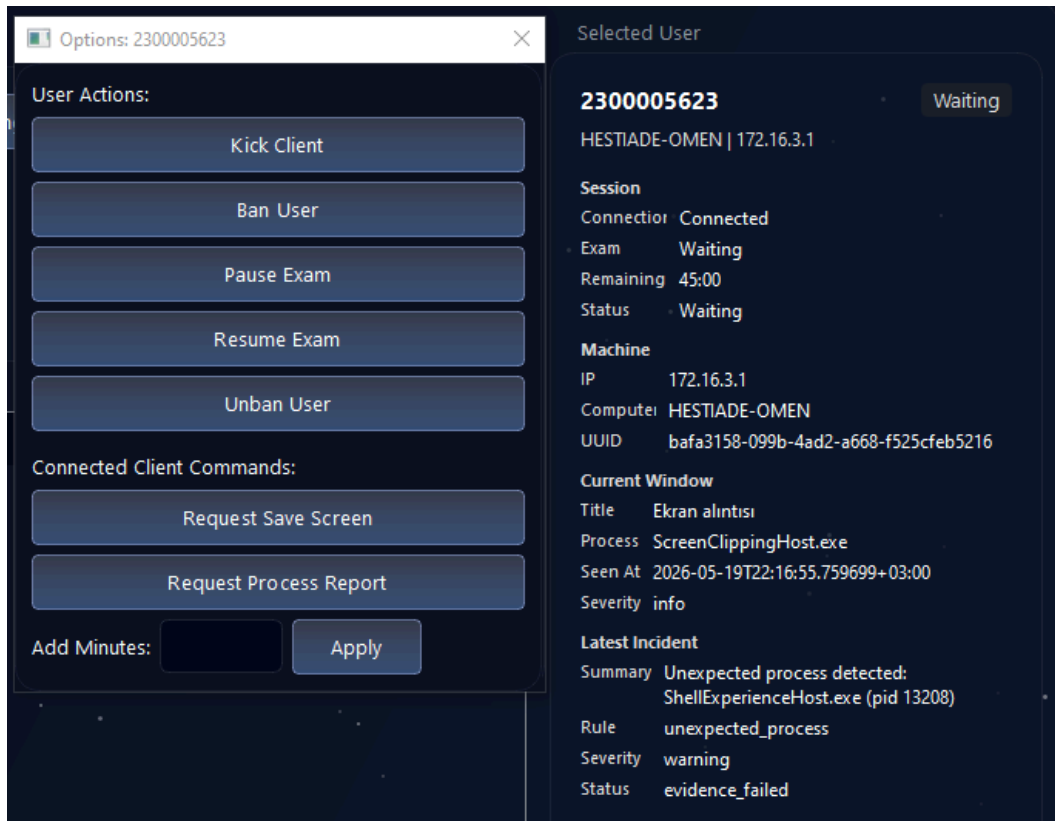


Early per-student detail card with circular timer and Quick Actions buttons.

AFTER

The integrated per-student detail is now split into two halves. A Selected User panel on the right-hand side reads the session, machine, current window and latest incident directly from the server state and refreshes on every event. A separate Options popup exposes user actions (Kick, Ban, Pause Exam, Resume Exam,

Unban), connected-client commands (Request Save Screen, Request Process Report) and a numeric Add Minutes field with an Apply button. Every action sent from the popup is wrapped by the same authorise-then-dispatch pipeline used by the CLI.



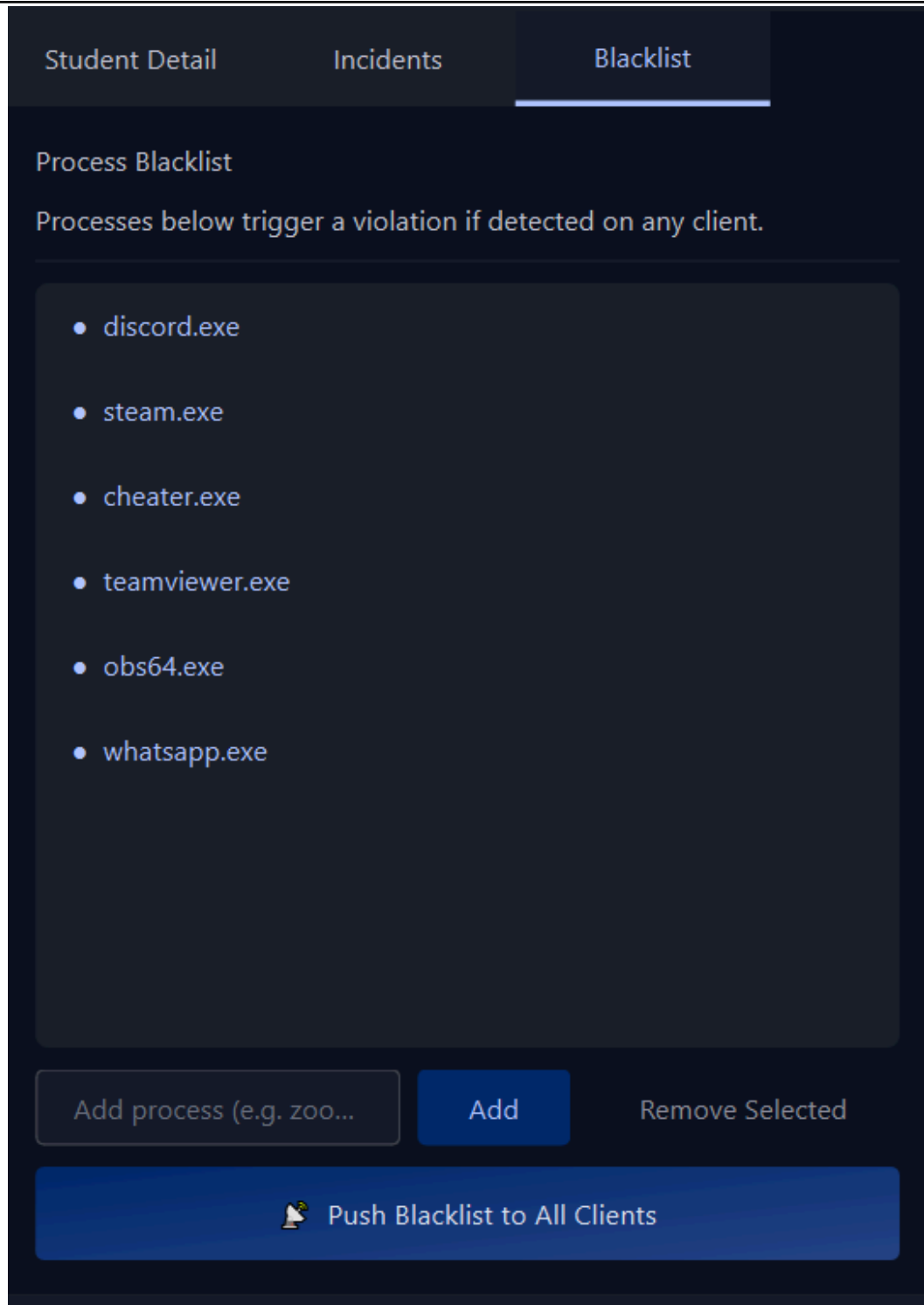
Integrated Options popup paired with the live Selected User panel.

Reason for change: Splitting the polished card into a read-only live panel and a modal action popup let the live data refresh continuously without interfering with the operator while they choose what to do. The Options popup centralises the action vocabulary so that the same command spelling is used by the dashboard, the CLI and the scripted test suite.

5. Policy and Blacklist Management

BEFORE

The first policy-management iteration was a single tab inside the student detail panel labelled Blacklist. It displayed a flat bulleted list of process names (discord.exe, steam.exe, cheater.exe, teamviewer.exe, obs64.exe, whatsapp.exe) with an Add process input, an Add button, a Remove Selected control and a single Push Blacklist to All Clients button. There was no notion of severity, no auto-pause toggle, no allowance for known good directories and no Window Rules. Every change applied immediately on push, without an export or rollback option.

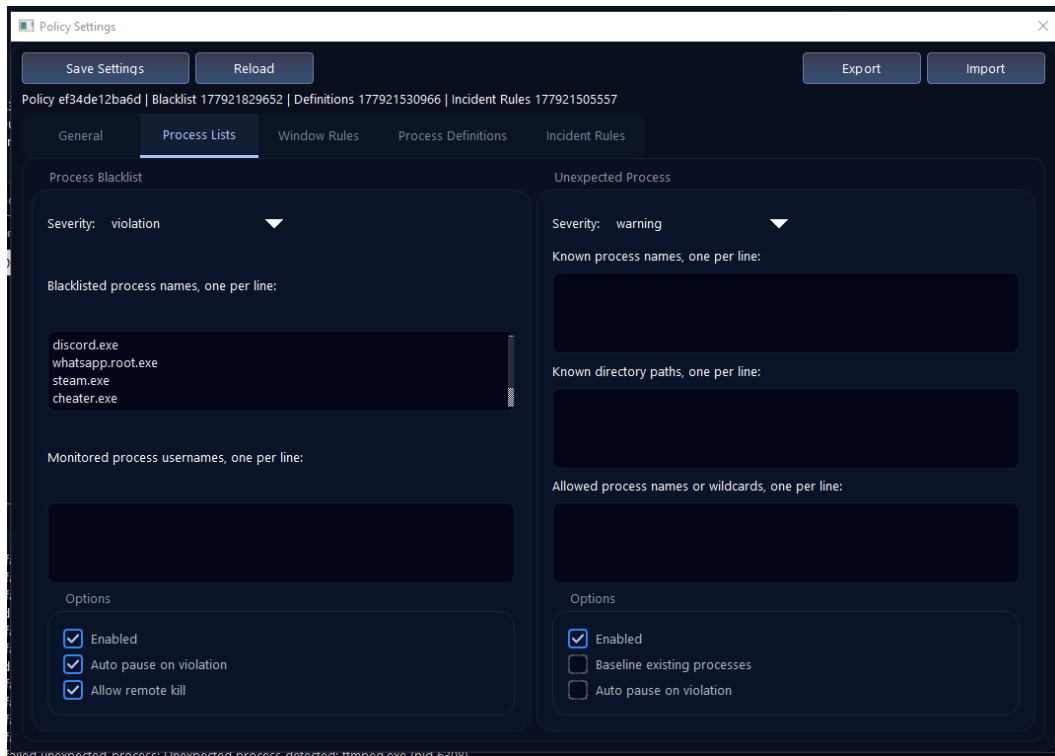


Early Blacklist tab embedded in the student detail panel.

AFTER

Policy management is now its own dedicated window with five tabs (General, Process Lists, Window Rules, Process Definitions, Incident Rules), a versioned policy banner across the top, and Save Settings / Reload / Export / Import buttons. Process Lists covers both Process Blacklist and Unexpected Process detection with

severity selectors, auto-pause-on-violation toggles, allow-remote-kill switches, baseline known-good directories and wildcard allowlists. Each policy change is staged in the window and only takes effect on Save Settings, which broadcasts the new policy version to every connected client.



Policy Settings window with multi-tab layout, versioned policy banner and Export / Import controls.

Reason for change: A flat blacklist could not express the difference between a warning, a violation that paused the exam and a violation that killed the offending process. The multi-tab policy window gives the operator a single place to encode the institutional exam-conduct rules and ship them to every client in one push.

6. Persistence and Audit Trail

BEFORE

In the first iterations the server kept all state in process memory: a dictionary keyed by login id held the user state, a list held the incident history and policy changes were applied directly to the running runtime. A crash or a manual restart discarded everything, including the timer progress, the latest incident summary and any pending evidence-upload retries. There was a single global incidents log but no per-session view; reconstructing a specific student's flow meant scanning the whole file by hand.

AFTER

The persistence layer is now a small set of JSON and JSONL files rooted under the server data directory, plus an audit-trail extension that adds five logical layers for after-exam review: an immutable session header per student, an append-only raw-event stream, an interpreted-violation layer with explicit pointers back to the raw events that triggered each violation, a per-day instructor-action log and a replace-in-place review-decision layer that preserves the full revision history. A reconstruct-timeline query merges the five layers into a chronologically ordered list keyed on the observation timestamp.

Reason for change: Loss of state on a crash is acceptable in a demo but not in an exam. Moving to atomic file persistence keeps recovery cost low, and the per-session layout means the post-exam reviewer never has to scan a global log to follow one student's flow.

7. Exam Control and Session State Machine

BEFORE

Exam control was originally a pair of boolean flags on each user record (`exam_started` and `exam_finished`) plus a banned boolean. Edge cases such as administrative pause, disconnect-driven pause and violation-driven pause all reduced to the same boolean shape, and any code path that needed the real state had to read all three flags together. There was no central place to express valid transitions, and a stray write could leave a student in an inconsistent combination of flags.

AFTER

Session state is now an eight-state finite state machine (Waiting, Running, Admin-Paused, Disconnected-Paused, Violation-Paused, Awaiting-Submission, Submitted and Banned) with a single mutation entry point that stamps a timestamp and a human-readable reason on every transition. The remaining-time snapshot is pinned on entry to a paused state and resumed from the same snapshot so a long pause does not drift. The operator command dispatcher routes both CLI stdin commands and dashboard actions through the same handler set so the two surfaces cannot diverge.

Reason for change: Replacing scattered boolean flags with a single authoritative state model removes a large class of inconsistency bugs and gives both the operator and the post-exam reviewer a single answer to the question "what state is this student in right now?".

8. Authentication and Identity

BEFORE

The first prototype carried its own credential exchange and a separate role-based check for the instructor. The instructor's credential was held as a constant inside the source code, and the only validation of a student login was that the supplied identifier appeared in an admission whitelist. There was no interaction with the institutional identity provider and no concept of a temporary-disable workflow.

AFTER

Authentication is now built around a single path that talks to the institutional CATS portal and (optionally) to Active Directory. The operator can temporarily disable either backend for a defined number of seconds (capped server-side) to handle outages, and any temporarily disabled login enters an admin-managed validation queue. The validation record stores the login identifier, the source IP and the disabled-auth modes but never the submitted password. A separate HMAC token gate prevents an unauthorised client from speaking to the server even if the credential check is bypassed.

Reason for change: Hard-coded credentials are not auditable and not revocable. Routing through CATS and Active Directory makes the identity decision align with the institution's existing controls; the temporary-disable workflow handles real-world failure modes without weakening the steady-state security model.

9. Post-Exam Review Surface

BEFORE

There was no formal post-exam review surface. After the exam, an academic-integrity claim could only be investigated by scanning the global incidents log by hand and correlating the timestamps with whatever screen-captures the operator had managed to save during the exam. Operator actions during the exam were

not recorded as a separate stream, so it was difficult to show after the fact whether a pause had been administrative or violation-driven.

AFTER

The post-exam reviewer now has three role-controlled HTTP endpoints. One returns a list of completed sessions. Another returns the merged chronological timeline for a single session, tagging each row with its kind (session_start, raw_event, violation, instructor_action, review or session_end). The third accepts a review decision (approve, flag, escalate) with a free-form note and a flags list, and every decision is appended as a new revision rather than overwriting the previous one.

Reason for change: Defensible academic-integrity decisions need an audit trail that explicitly distinguishes observation, interpretation and intervention. The three-endpoint surface lets a reviewer answer the only question that matters: what happened, how was it interpreted at the time, and what did the operator do about it.

9 References

- Tanenbaum, A. S., & Wetherall, D. J. (2010). Computer Networks (5th ed.). Pearson.
- Fette, I., & Melnikov, A. (2011). The WebSocket Protocol. RFC 6455. IETF.
- Fielding, R., & Reschke, J. (2014). Hypertext Transfer Protocol: Message Syntax and Routing. RFC 7230. IETF.
- Eastlake, D., & Hansen, T. (2011). US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF). RFC 6234. IETF.
- aiohttp documentation. <https://docs.aiohttp.org>
- psutil documentation. <https://psutil.readthedocs.io>
- cryptography (Fernet) documentation. <https://cryptography.io/en/latest/fernet/>
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering.
- ISO/IEC/IEEE 12207:2017 — Systems and software engineering — Software life cycle processes.
- Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446. IETF.
- Krawczyk, H., Bellare, M., & Canetti, R. (1997). HMAC: Keyed-Hashing for Message Authentication. RFC 2104. IETF.
- Bormann, C., & Hoffman, P. (2020). Concise Binary Object Representation (CBOR). RFC 8949. IETF.
- Crocker, D., & Overell, P. (2008). Augmented BNF for Syntax Specifications: ABNF. RFC 5234. IETF.
- Hartman, S. (2014). Web Origin Concept. RFC 6454. IETF.

- OWASP. (2024). OWASP Top Ten 2021 — A guide to web application security risks. <https://owasp.org/Top10/>
- OWASP. (2024). OWASP Application Security Verification Standard (ASVS) 4.0. <https://owasp.org/www-project-application-security-verification-standard/>
- NIST. (2017). Digital Identity Guidelines, SP 800-63-3. National Institute of Standards and Technology.
- NIST. (2020). Zero Trust Architecture, SP 800-207. National Institute of Standards and Technology.
- Python Software Foundation. (2024). The Python Language Reference, Version 3.13. <https://docs.python.org/3.13/>
- Python Software Foundation. (2014). PEP 8 — Style Guide for Python Code. <https://peps.python.org/pep-0008/>
- van Rossum, G., Lehtosalo, J., & Langa, Ł. (2014). PEP 484 — Type Hints. <https://peps.python.org/pep-0484/>
- Salgado, P., & Selivanov, Y. (2019). PEP 604 — Allow writing union types as X | Y. <https://peps.python.org/pep-0604/>
- Coghlan, N., & Stinner, V. (2019). PEP 538 — Coercing the legacy C locale to a UTF-8 based locale. <https://peps.python.org/pep-0538/>
- websockets: a library for building WebSocket servers and clients in Python. <https://websockets.readthedocs.io>
- PySide6: Official Python bindings for Qt 6 (LGPL). <https://doc.qt.io/qtforpython-6/>
- Pillow: Python Imaging Library fork. <https://pillow.readthedocs.io>
- Beautiful Soup: HTML and XML parsing library. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- Requests: HTTP for Humans. <https://requests.readthedocs.io>
- FFmpeg Project. (2024). FFmpeg documentation. <https://ffmpeg.org/documentation.html>
- FFmpeg Project. gdigrab — Win32 GDI-based screen capture device. <https://ffmpeg.org/ffmpeg-devices.html#gdigrab>
- SQLite Consortium. (2024). SQLite documentation. <https://www.sqlite.org/docs.html>
- Sommerville, I. (2015). Software Engineering (10th ed.). Pearson.
- Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.
- Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Hunt, A., & Thomas, D. (2019). The Pragmatic Programmer: Your Journey to Mastery (20th Anniversary ed.). Addison-Wesley.
- Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems (2nd ed.). O'Reilly Media.
- Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.
- Beck, K. (2003). Test-Driven Development: By Example. Addison-Wesley.
- Anderson, R. (2020). Security Engineering: A Guide to Building Dependable Distributed Systems (3rd ed.). Wiley.

- Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson.
- International Center for Academic Integrity. (2021). *The Fundamental Values of Academic Integrity* (3rd ed.). <https://academicintegrity.org/>
- Bilen, E., & Matros, A. (2021). Online cheating amid COVID-19. *Journal of Economic Behavior & Organization*, 182, 196–211.
- Microsoft. (2024). Windows API Reference — Win32 API documentation. <https://learn.microsoft.com/en-us/windows/win32/api/>
- Microsoft. (2024). Active Directory Domain Services overview. <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/active-directory-domain-services>
- Microsoft. (2024). Lightweight Directory Access Protocol (LDAP) API. https://learn.microsoft.com/en-us/windows/win32/api/_ldap/
- EmpCloud. emp-monitor: Open-source employee activity monitoring tool. <https://github.com/EmpCloud/emp-monitor>
- Gorav22. AI-Proctor-Exam-Monitor: AI-assisted online exam proctoring monitor. <https://github.com/Gorav22/AI-Proctor-Exam-Monitor>
- Demir, C. Reference projects on GitHub. <https://github.com/caner-demir/>
- Online Exam Monitoring System. *International Journal of Engineering Development and Research*, Volume 8, Issue 3, 2020. <https://rjwave.org/ijedr/papers/IJEDR2003018.pdf>
- Epoptes: Open-source classroom management and supervision tool for Linux. <https://github.com/epoptes/epoptes>
- henrypp. simplewall: Lightweight Windows firewall with per-application rules. <https://github.com/henrypp/simplewall>
- ActivityWatch: Cross-platform automated time-tracking and activity logging. <https://github.com/ActivityWatch/activitywatch>
- Veyon: Cross-platform classroom-management and computer-monitoring framework. <https://github.com/veyon/veyon/>
- Mythware E-Classroom Management Software (community study materials). <https://github.com/114514NotAvailable/Mythware-E-Classroom-Management-Software-v2.7.17331-Cracked>